Course 4509

Introduction to Python for Data Analytics

by

Mary Flynn

LEARNING TREE INTERNATIONAL

Copyright

Introduction and Overview

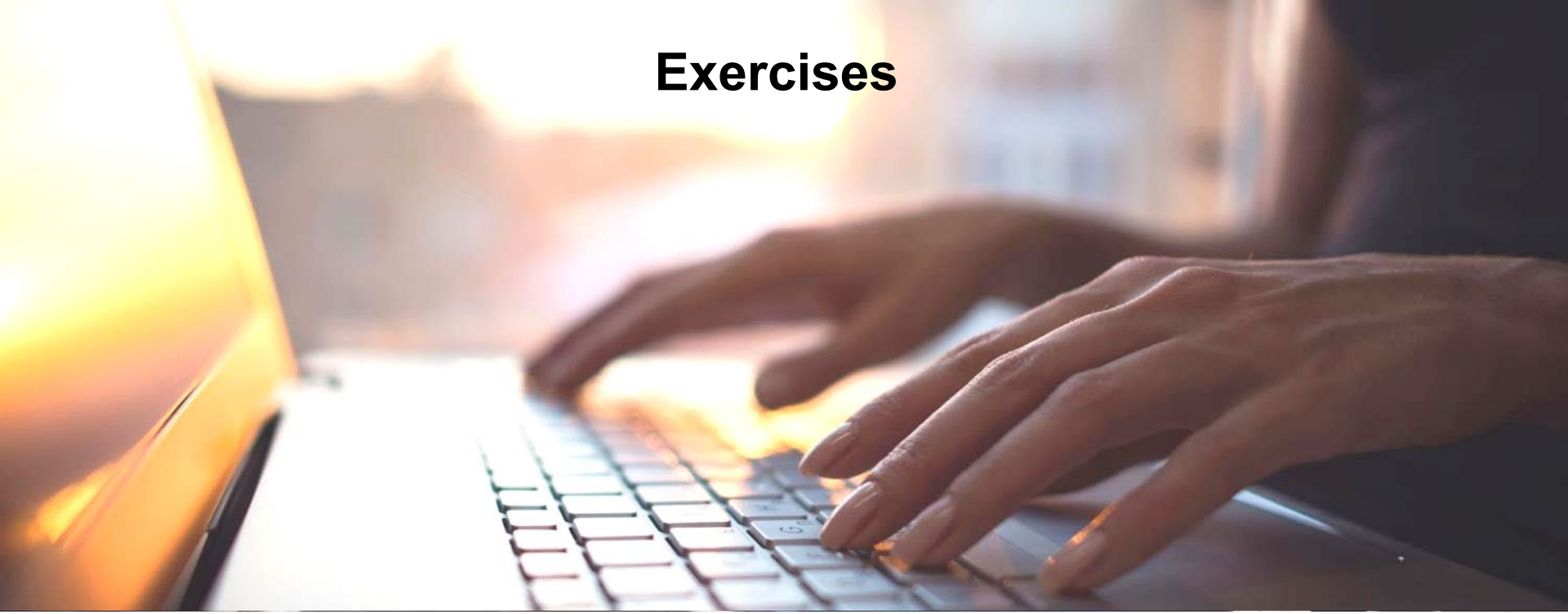
LEARNING TREE INTERNATIONAL

Course Objectives

- Review Jupyter as a tool for working with Python for data analytics
- Import data from varied sources into Python
- Manipulate and transform data in preparation for data mining
- Pre-process unstructured data for data analytic tasks
- Optional: introduce machine learning in Python

Course Contents

Exercises



- Exercises are located throughout the course
- Help reinforce your learning
- The course uses an on-line Virtual Machine (VM) for various exercises

LEARNING TREE INTERNATIONAL

Objectives

- Introduce the programming language Python
- Explore the Jupyter environment
- Import external data sets into Python

Contents

Introduction to Python

- Importing Data Sets Into Python
- Hands-On Exercise 1.1

Introduction to Python

- **Free, open-source software, with a vibrant community**
- **Python is a general-purpose language**
  - No single environment to work in
  - No single way of using it
  - Can make it harder for beginners to find their way
- **Widely used in academia and for high-performance scientific projects**
- **Does not come with a pre-bundled set of modules for analytic computing**
- **Uses external libraries, usually written in faster languages (e.g., C, C++, etc.)**
  - The main libraries used are *NumPy*, *SciPy* and *Matplotlib*

Python Stacks

- **The easiest way to install a Python stack is to download one of these Python distributions, which include all the key packages:**

Distribution	Description	Supports
Anaconda	Free distribution	Linux, Windows, and Mac
Enthought Canopy	Free and commercial versions	Linux, Windows, and Mac
Python(x,y)	Free distribution based around the Spyder IDE	Windows only
WinPython	Free distribution	Windows only
Pyzo	Free distribution based on Anaconda and the IEP interactive development environment	Linux, Windows, and Mac

Python IDEs: Jupyter Notebooks

- **An open-source web application**
  - Provides you with an easy-to-use, interactive data science environment across many programming languages
- **Not a traditional IDE**
  - Can also act as a presentation or education tool
  - Allows creation and sharing of documents that contain live code, equations, visualizations, and narrative text
    - Graphs can be shown in the same document as the code
    - Allows adding of HTML components (e.g., images, videos)

- **Final work can be exported to PDF and HTML files**

Jupyter Notebook Document

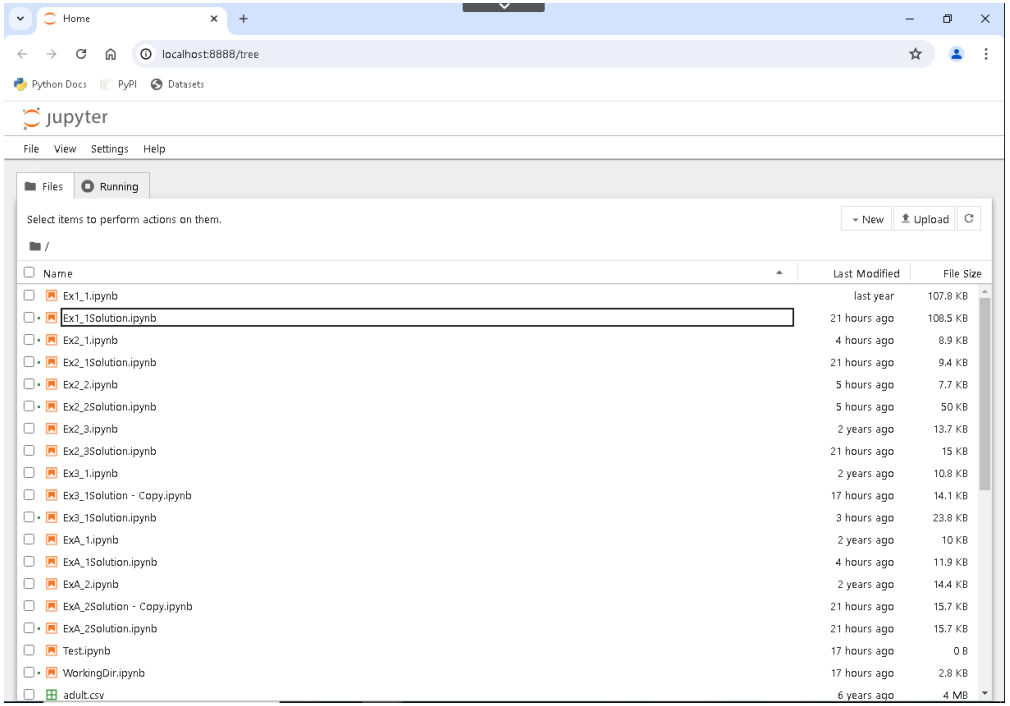
- **Notebooks are documents produced by the Jupyter Notebook App**
  - Contains both computer code (e.g., Python) and rich text elements (paragraph, equations, figures, links, etc.)
  - Human-readable
    - Can contain a description of the analysis and the results (e.g., narrative, images, etc.)

Jupyter Notebook App

- **Allows editing and execution of notebooks via a web browser**
  - Can be executed on local desktop, on remote server, or through the internet
- **Has a "Dashboard" showing local files, which allows opening of Notebooks or shutting down of their kernels**
  - A *kernel* is a "computational engine" that executes the code contained in a Notebook document
    - Here the *kernel* executes Python code, but *kernels* for other languages exist
- **When Notebook is launched, the associated *kernel* is launched**
  - When the Notebook is executed (either cell-by-cell or all cells), the *kernel* performs the computation and produces the results
    - Depending on the type of computations, the *kernel* may consume significant CPU and RAM
    - RAM is released when the *kernel* is shut down

The Notebook Dashboard

- **Shown when you first launch Jupyter Notebook App**
- **Used to open notebook documents**
- **Also used to manage the running kernels**
  - View or shutdown
- **Similar to a file manager**
  - Used to navigate folders
  - Used to rename or delete files



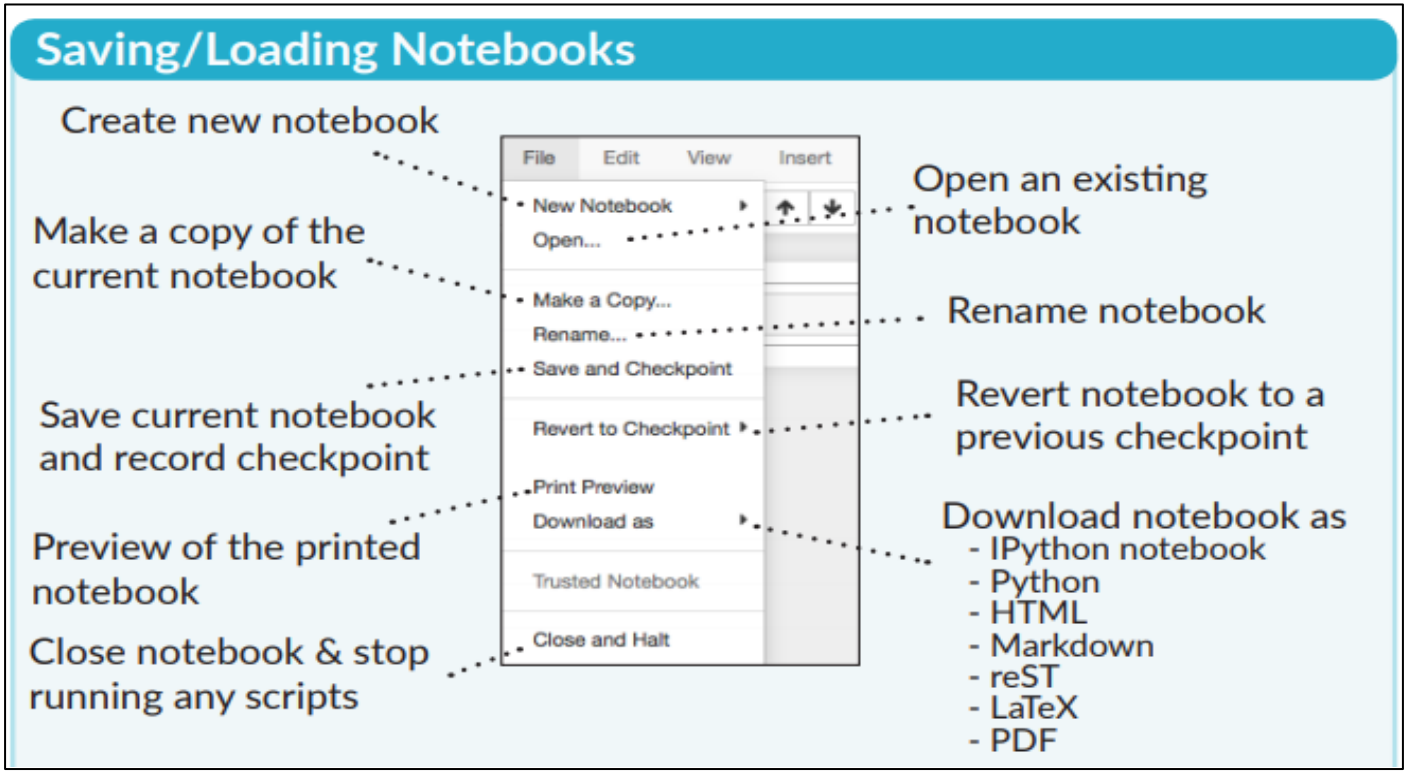
Jupyter Notebook App

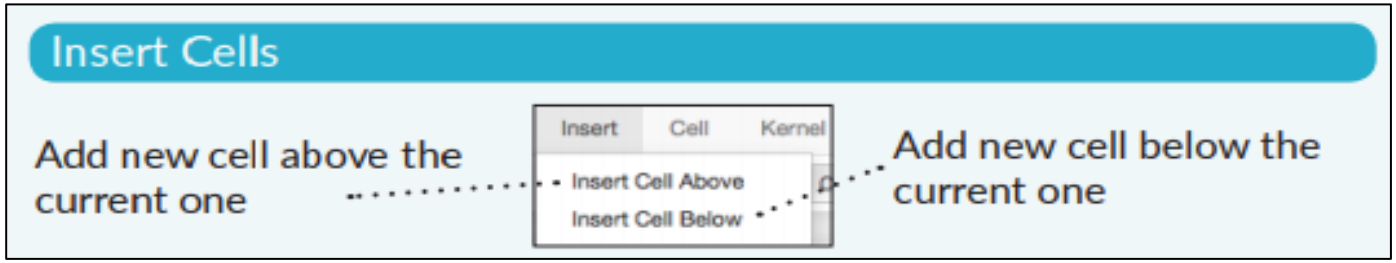
- **The Jupyter Notebook App can access only files within its start-up folder (including any sub-folder)**
- **The Jupyter Notebook App is a server that appears in your browser at a default address (`http://localhost:8888`)**
- **To shut it down, you need to close the associated terminal**
  - Closing the browser (or the tab) will not close the Jupyter Notebook App
- **When a notebook is opened, its "computational engine" (*kernel*) is automatically started**
  - Closing the notebook browser tab, will not shut down the kernel
  - It will keep running until is explicitly shut down
- **To shut down a kernel, go to the associated notebook and click Close and Shut Down Notebook**
  - Alternatively, the Notebook Dashboard has a tab named *Running* that shows all the running notebooks (i.e., kernels)
    - Allows shutting them down by right-clicking and selecting the *Shutdown Kernel* option

Executing a Notebook

- **Double-Click the notebook name and it will open in a new browser tab**
- **A notebook document can be executed one cell at a time by:**
  - `<Ctrl><Enter>`: execute cell
  - `<Shift><Enter>`: execute cell and highlight/create cell below
- **The entire notebook can be executed by clicking Cell -> Run All**
- **Changes to notebooks are automatically saved every few minutes**
- **Care should be taken not to open the same notebook document on many tabs**
  - Edits on different tabs can overwrite each other

Saving/Loading Notebooks



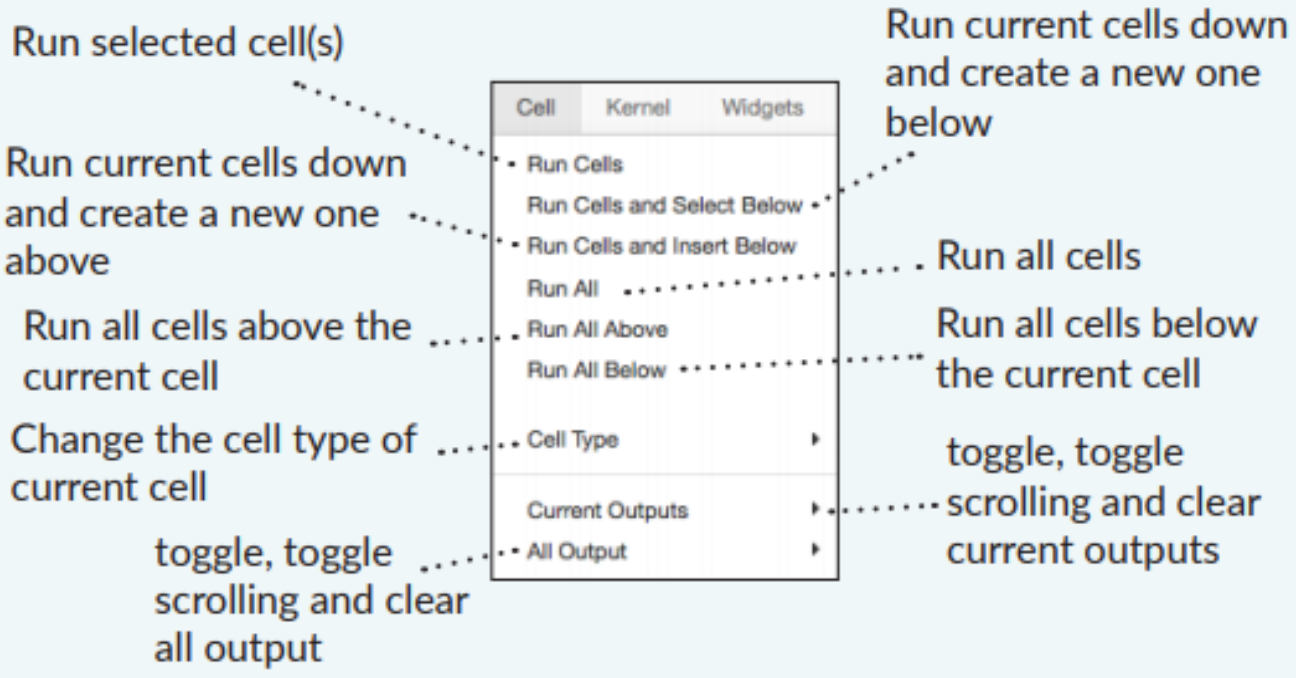


Notebook Cells

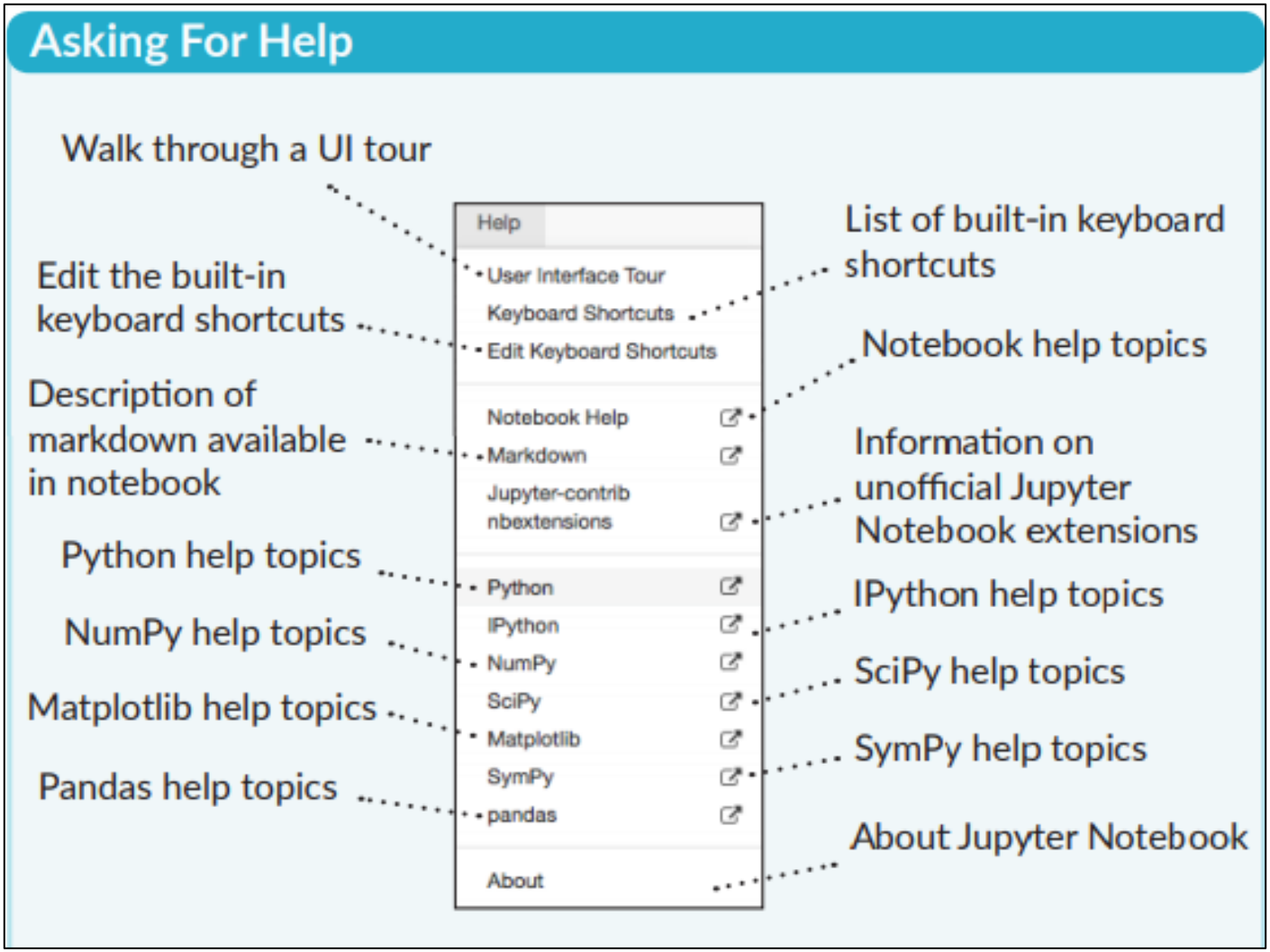
Writing Code And Text

Code and text are encapsulated by 3 basic cell types: markdown cells, code cells, and raw NBConvert cells.

Executing Cells



Accessing Help



Contents

External Libraries

- NumPy: provides numerical arrays and routines for manipulation
  http://www.numpy.org/
- SciPy: provides high-level data processing routines, such as regression, etc.
  https://www.scipy.org/
- Matplotlib: provides comprehensive 2-D visualizations
  https://matplotlib.org/
- Pandas: provides high-performance, easy-to-use data structures and data analysis tools
  http://pandas.pydata.org/
- Scikit-Learn: provides tools for data mining and data analysis
  http://scikit-learn.org/stable/
- IPython: an enhanced interactive console
  https://ipython.org/

Importing Libraries

- **Once libraries are installed, they can be used by importing them at the beginning of the code**
  - Must be done for every new notebook used
  - Functions within a module can then be referenced using dot notation
  - Module names may also be changed using aliases

```
In [1]: import math

        math.sqrt(9)

Out[1]: 3.0
```

```
In [4]: import math as m

        m.sqrt(9)

Out[4]: 3.0
```

- **Functions from a module may be referred to directly when importing**
  - This eliminates the need for dot notation

```
In [3]: from math import sqrt

        sqrt(25)

Out[3]: 5.0
```

Importing Data Into Python

- **Data can be imported from many different sources into Python; for example:**
  - CSV files/Excel files
  - Websites
  - Relational databases and data warehouses
  - NoSQL data stores (e.g., MongoDB)
  - XML documents
  - JSON files
  - Library option also allows many other types to be processed

CSV = comma-separated value
JSON = JavaScript Object Notation
XML = extensible markup language

Loading Data Into Python: CSV files

- **The `pandas` library comes with useful abstractions for dealing with datasets**
- **To load data from an external (`.csv`) file into a pandas `DataFrame`, use the `read_csv()` method:**

```
import pandas as pd
```

```
iris = pd.read_csv('iris.csv')
```

```
iris
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

- **The `pandas` library also comes with a lot of functionality for ETL**
  - Enables succinct expression of data transformations and provides functions to load, unify, and store data from different sources and formats

Importing Data From a Relational Database

- **The SQLAlchemy package provides a way of interacting with databases**

```
import sqlalchemy as sql
```

```
connect_string = 'mysql://mary:password@localhost/classicmodels'

sql_engine = sql.create_engine(connect_string)
```

```
query = "select * from customers"
df = pd.read_sql_query(query, sql_engine)
```

```
df
```

- **Connecting to an Oracle Database—(`cx_oracle` is a driver) (default drivers are used where this is not specified)**

```
sql_engine =
create_engine('oracle+cx_oracle://scott:tiger@tnsname')
```

- **Connecting to a SQLServer Database—(pyodbc is a driver)**

```
sql_engine = create_engine('mssql+pyodbc://scott:tiger@mydsn')
```

Loading Data Into Python: Web Scraping

- **The Pandas library has a built-in method to scrape tabular data from html pages called `read_html()`:**

Interpret the 1st row as column headers

```
In [55]: tables = pd.read_html("https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population", header=0)

         tables[1]
```

Out[55]:

	Rank	Country(or dependent territory)	Population	Date	% of worldpopulation	Source
0	1	China[Note 2]	1393570000	August 18, 2018	18.2%	Official population clock
1	2	India[Note 3]	1335800000	August 18, 2018	17.5%	Official population clock
2	3	United States[Note 4]	327675000	August 18, 2018	4.29%	Official population clock
3	4	Indonesia	265015300	July 1, 2018	3.47%	Official annual projection
4	5	Brazil	209463000	August 18, 2018	2.74%	Official population clock

Display the 2nd table retrieved from the webpage

Loading External Data Into Python: Web Scraping

- **Many websites may have data you would like to read programmatically**
  - Use caution—attempting to read pages too quickly can get your IP address blocked
- **HTML parsing in Python is simplified with use of a library called *Beautiful Soup***
  - Allows navigating and searching of the parse tree
- **Example: To display all the URLs on a web page:**

```
from bs4 import BeautifulSoup
import requests
```

```
r = requests.get("http://en.wikipedia.org/wiki/List_of_countries_by_population ")
```

```
soup = BeautifulSoup(r.content, "html.parser")
```

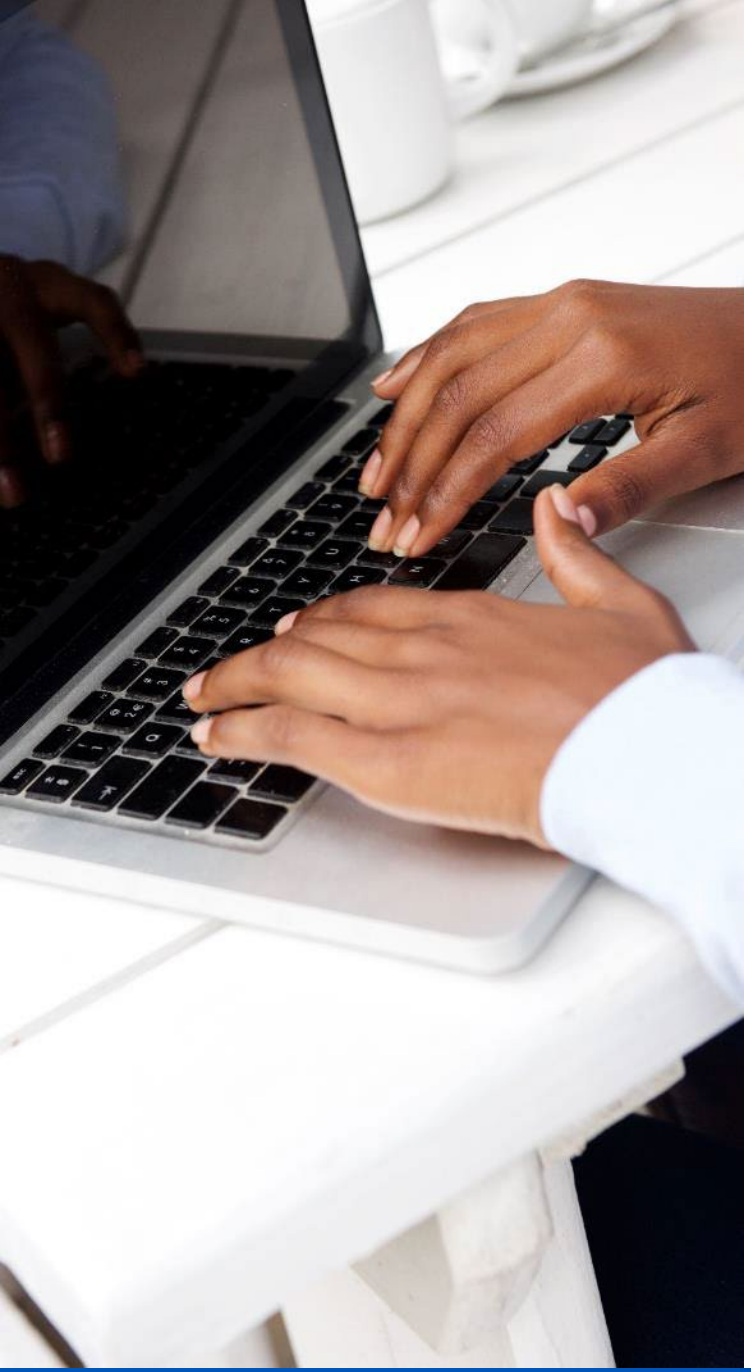
```
for link in soup.find_all('a'):
    print(link.get('href'))
```

HTML = hypertext markup language

Contents

- Introduction to Python
- Importing Data Sets Into Python

Hands-On Exercise 1.1



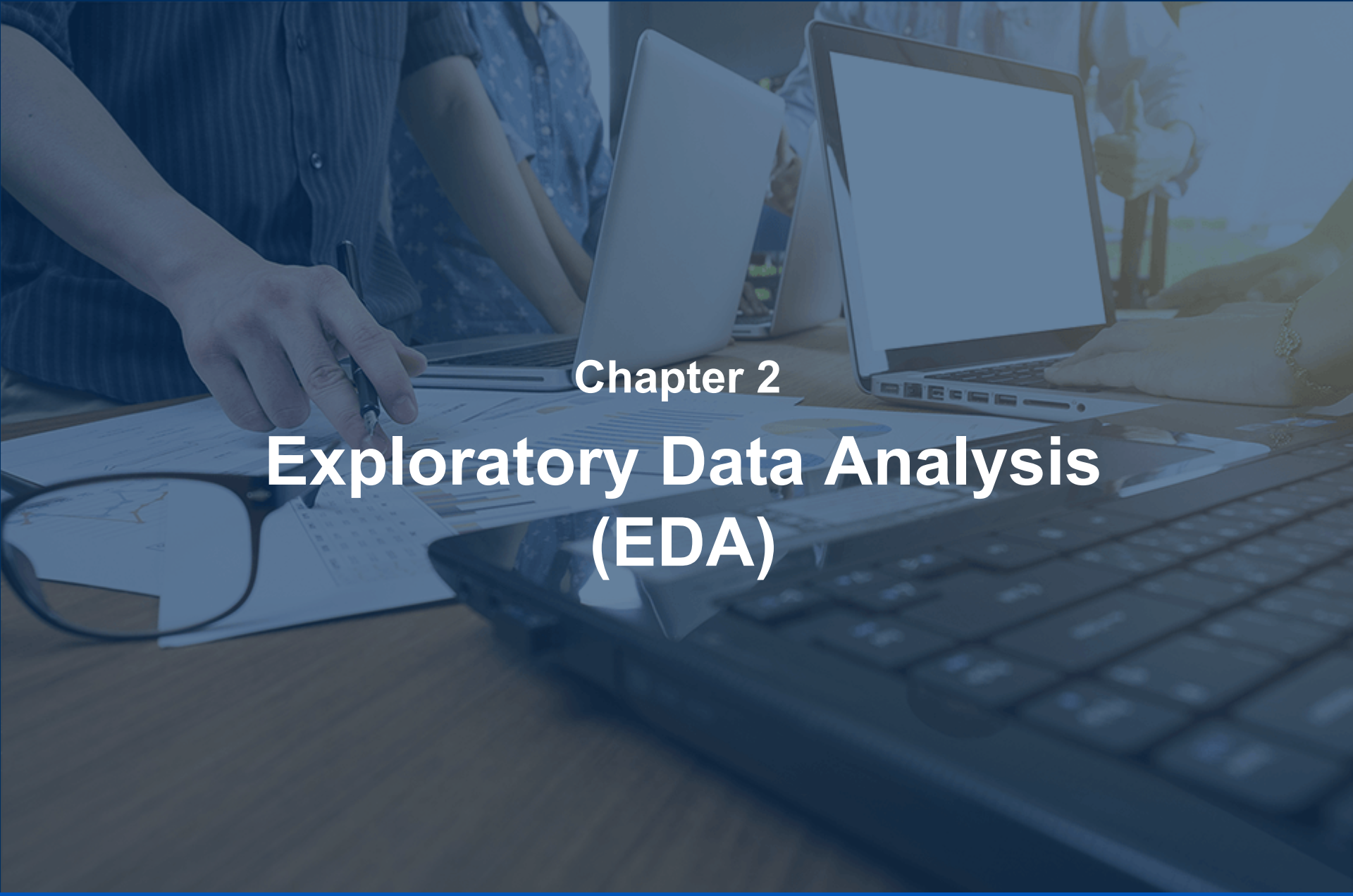
Hands-On Exercise 1.1

Jupyter Exercise Dashboard

In your Jupyter Exercise Dashboard, please refer to Hands-On Exercise 1.1: Sourcing and Loading Data in Python

Objectives

- Introduce the programming language Python
- Explore the Jupyter environment
- Import external data sets into Python



Chapter 2

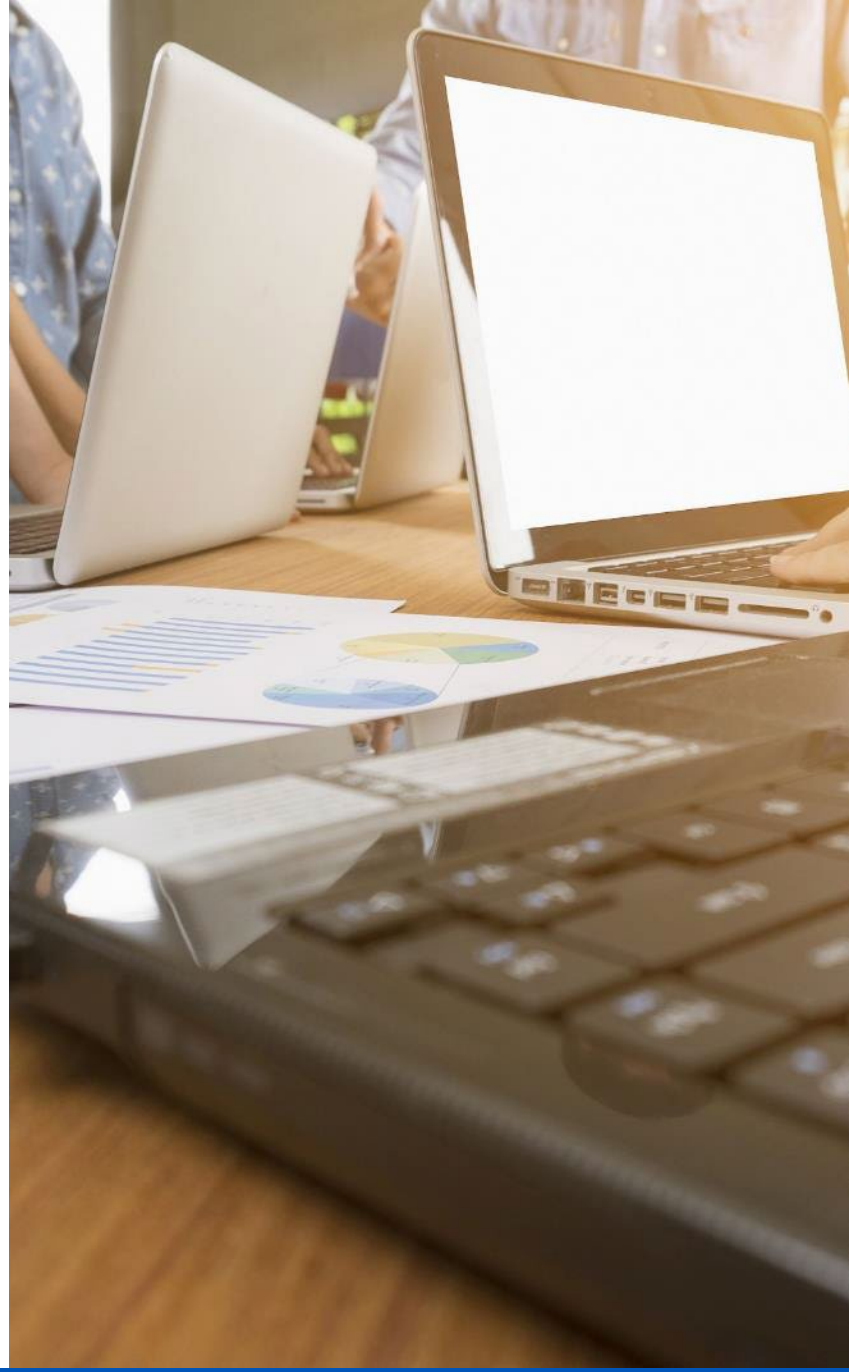
Exploratory Data Analysis (EDA)

Objectives

- Explore data sets with Python
- Employ data visualization as an aid to understanding data sets
- Prepare data for analysis

Contents

Working With Data in Python



Data Types

- **There are many basic data types in Python**

Data Type	Usage
Object	Text
`int64`	Integer numbers
`float64`	Floating point numbers
`datetime64`	Date and time values
`timedelta[]`	Differences between two date times
`bool`	True/False values

- **Data types can be assigned to variables using the assignment operator =**
  - For example, `varA = 2`
    - `varB = 'Hello'`
  - Strings
    - Can be defined with either single quotes (') or double quotes (")
    - Can concatenate strings using the + operator

Exploratory Analysis Using the pandas Library

- **pandas is the most widely used library in Python for data munging/analysis**
  - Provides flexible data structures that make working with data easier and more intuitive
- **Allows for practical, real-world data analysis in Python**
- **There are two primary data structures in pandas**
  - Easier to work with than other data structures in Python (lists, dictionaries, etc., requiring `For… Loops`)
    - `Series` (one-dimensional)
    - `DataFrame` (two-dimensional)
- **There are other data structures in Python for storing data values in memory, but we will focus mainly on the above two in this course**
  - Lists (ordered sequences of values)
  - Tuples (ordered, immutable sequences of values)
  - Sets (unordered bags of values)
  - Dictionaries (unordered bags of key-value pairs)

Series

- **A one-dimensional array capable of holding any data type**
  - All values in a single series must be of the same data type

```
Out[26]: 0     1.331587
         1     0.715279
         2    -1.545400
         3    -0.008384
         4     0.621336
         5    -0.720086
         6     0.265512
         7     0.108549
         8     0.004291
         9    -0.174600
         10    0.433026
```

DataFrame

- **A two-dimensional array stored as a collection of series**
- **A tabular data structure comprised of rows and columns**
  - Similar to a spreadsheet or database table
- **A `DataFrame` can be used to represent an entire dataset**
  - The most commonly used structure in data exploration

```
iris = pd.read_csv('iris.csv')
iris
```

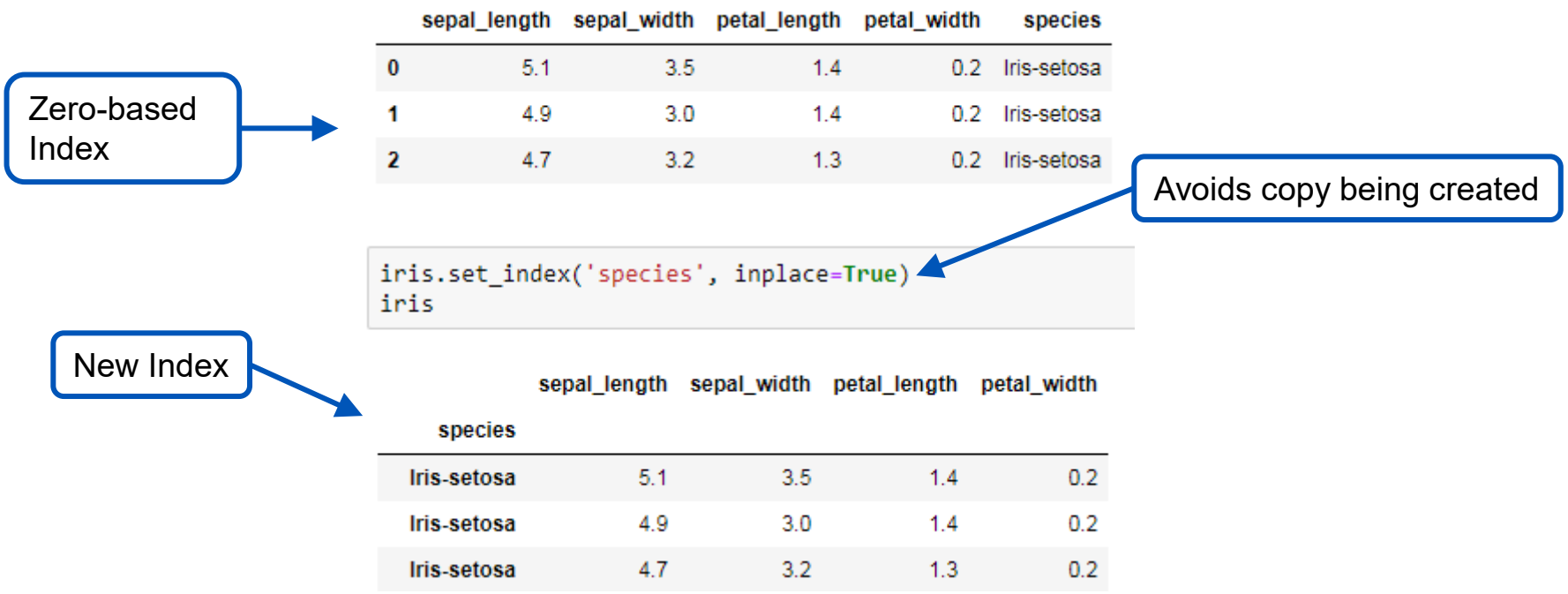
Column names

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa

Index

DataFrame Indexes

- **By default the `DataFrame` will automatically create a zero-based index for the dataset**
  - Here the iris dataset has been loaded with the default zero-based index
  - To make a column from the dataset, the index, use the `.set_index()` method



	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa

species	sepal_length	sepal_width	petal_length	petal_width
Iris-setosa	5.1	3.5	1.4	0.2
Iris-setosa	4.9	3.0	1.4	0.2
Iris-setosa	4.7	3.2	1.3	0.2

Setting an Index on Data Load

- **The index may also be set when the data is being loaded using the `index_col` parameter**

```
iris = pd.read_csv('iris.csv', index_col=4)
iris
```

species	sepal_length	sepal_width	petal_length	petal_width
Iris-setosa	5.1	3.5	1.4	0.2
Iris-setosa	4.9	3.0	1.4	0.2
Iris-setosa	4.7	3.2	1.3	0.2

This indicates that the 5th column in the `.csv` file should be used as the index

(Index column positions are zero-based)

Examining the Data's Structure

- **A `DataFrame` has methods and attributes which can be called on, to examine the data's structure**
  - Methods are called with parentheses
  - Attributes are called without parentheses
- **To examine the columns of the iris dataset, use the `.columns` attribute**

```
iris.columns

Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='object')
```

- **The dimensions of the dataset are given by the `.shape` attribute**
  - This example shows that the iris dataset has 150 rows and 5 columns

```
iris.shape
(150, 5)
```

Examining the Data's Structure

- **To examine the index of a `DataFrame`, use the index attribute**

```
iris.index
```

```
RangeIndex(start=0, stop=150, step=1)
```

- **To examine the data types of the `DataFrame`, use the `.dtypes` attribute or for a summary of the `Dataframe's` structure, use the `.info()` method**

```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
sepal_length    150 non-null float64
sepal_width     150 non-null float64
petal_length    150 non-null float64
petal_width     150 non-null float64
species         150 non-null object
dtypes: float64(4), object(1)
memory usage: 5.9+ KB
```

- **When a dataset is loaded into a `pandas DataFrame`, it will infer the data types**
  - Numbers will be stored in a numeric datatype (i.e., `int64` or `float64`)
  - Text will be stored in an object datatype, etc.

Displaying the Data in a DataFrame

- **To display the contents of the DataFrame, use the print() function:**

```
print(iris)
```

```
     sepal_length  sepal_width  petal_length  petal_width         species
0             5.1          3.5           1.4          0.2     Iris-setosa
1             4.9          3.0           1.4          0.2     Iris-setosa
2             4.7          3.2           1.3          0.2     Iris-setosa
3             4.6          3.1           1.5          0.2     Iris-setosa
4             5.0          3.6           1.4          0.2     Iris-setosa
5             5.4          3.9           1.7          0.4     Iris-setosa
6             4.6          3.4           1.4          0.3     Iris-setosa
```

- **In Jupyter notebooks, simply typing the DataFrame name results in nicely formatted outputs**
  - Displays only 20 columns by default for wide data DataFrames, and only 60 or so rows, truncating the middle section

```
iris
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa

Previewing the Data

- A small selection of rows from the `DataFrame` can be previewed with the `head()` and `tail()` methods

```
iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
iris.tail()
```

	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

Retrieving Data Subsets Using the [ ] Indexer

- **A column may be retrieved by specifying the name within single square parentheses**
  - Single brackets return a series

```
iris['sepal_length']
```

```
0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
5      5.4
```

- **To retrieve a subset of the column rows, a range may be specified as follows:**

```
iris['sepal_length'][0:3]
```

```
0    5.1
1    4.9
2    4.7
Name: sepal_length, dtype: float64
```

Retrieving Data Subsets Using the [ ] Indexer

- **To retrieve multiple columns from the `DataFrame`, a list of values should be passed to the indexer, which results in double square brackets being used**
  - This will return a `DataFrame`
    - Single brackets would result in an error

```
iris[['sepal_length','sepal_width']]
```

	sepal_length	sepal_width
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6

Using the [ ] Indexer With a Filtering Condition

- **To filter rows, specify a Boolean condition that must be met before rows are returned**
  - Returns a series of True and False data with which to filter rows
  - Multiple conditions may be specified with an & symbol

```
iris[iris['sepal_length']>7.6]
```

	sepal_length	sepal_width	petal_length	petal_width	species
117	7.7	3.8	6.7	2.2	Iris-virginica
118	7.7	2.6	6.9	2.3	Iris-virginica
122	7.7	2.8	6.7	2.0	Iris-virginica
131	7.9	3.8	6.4	2.0	Iris-virginica
135	7.7	3.0	6.1	2.3	Iris-virginica

- **To restrict columns being retrieved, specify columns required as follows:**

```
iris[iris['sepal_length']>7.6][['species','sepal_length']]
```

	species	sepal_length
117	Iris-virginica	7.7
118	Iris-virginica	7.7
122	Iris-virginica	7.7
131	Iris-virginica	7.9
135	Iris-virginica	7.7

Retrieving Data Subsets Using the `.iloc[]` Indexer

- **Selections using the `.iloc[]` indexer are based on the numeric position of the row in the `DataFrame`**
- **Syntax:**

`df_name.iloc[<row selection>, <column selection>]`	
• Single row position e.g., 2 (i.e., 3rd row) • List of row positions e.g., [2,3,4] • Range/slice of rows e.g., [2:5]	• Optional • Single column position e.g., 1 • List of column positions e.g., [0,1,2] • Range/Slice of column positions e.g., [1:4]

- **Examples:**

```
iris.iloc[3]          # Single row
iris.iloc[[3,4]]      # Multiple rows
iris.iloc[3,1]     # Single row, single column
iris.iloc[3,[1,2]]     # Single row, multiple columns
iris.iloc[[3,4],[1,2]]     # multiple rows, multiple columns
iris.iloc[3,1:3]     # Single row, range of columns
```

Examples Using the `.loc[]` Indexer

Country	Rank	Population	Date	% of worldpopulation	Source
Indonesia	4	265015300	July 1, 2018	3.47%	Official annual projection
Brazil	5	208721442	August 21, 2018	2.73%	Official population clock
Pakistan	6	201174754	August 21, 2018	2.63%	Official population clock
Nigeria	7	197227340	August 21, 2018	2.58%	Official population clock
Bangladesh	8	167008838	August 21, 2018	2.18%	Official population clock

```
Countries.loc['Brazil']                 # Single row
Countries.loc[['Indonesia','Brazil']]  # Multiple rows
Countries.loc[Countries['Rank'] == '3'] # Row based on filter condition
Countries.loc['Brazil','Rank']          # Single row and single column
Countries.loc['Brazil',['Rank','Population']]  # Single row, multiple columns
Countries.loc[['Indonesia','Brazil'],['Rank','Population']] # Multiple rows,
                                                            #   multiple columns
Countries.loc['Brazil','Rank':'Source']   # Single row, range of columns
```

Dropping Columns from a DataFrame

- Columns are dropped from a DataFrame by specifying the axis parameter as 1

```
iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
iris.drop('species',axis=1).head()
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Lists

- **There are many other data structures in Python for storing data beyond `pandas DataFrames`, i.e., tuples, sets, dictionaries**
- **One such structure is a `list`**
  - `Lists` are quite flexible data structures
  - Can be used to store any combination of data values together

```
new_list = ['a', 'b', 'python', 'c', 'data science']
new_list
```

```
['a', 'b', 'python', 'c', 'data science']
```

```
new_list[0]
```

```
'a'
```

```
new_list[4]
```

```
'data science'
```

```
new_list[-1]
```

```
'data science'
```

```
new_list[-3]
```

```
'python'
```

```
new_list[1:3]
```

```
['b', 'python']
```

Dictionaries

- **A *dictionary* is an unordered set of key-value pairs**
  - When a key is added to a dictionary, a value must also be added for that key
    - Can be changed later
    - Optimized for retrieving the value when the key is known, but not the other way around
  - Cannot have duplicate keys
    - Assigning a value to an existing dictionary key simply replaces the old value with the new one

```
new_dict = {'language': 'python', 'database': 'mysql'}
new_dict
```

```
{'database': 'mysql', 'language': 'python'}
```

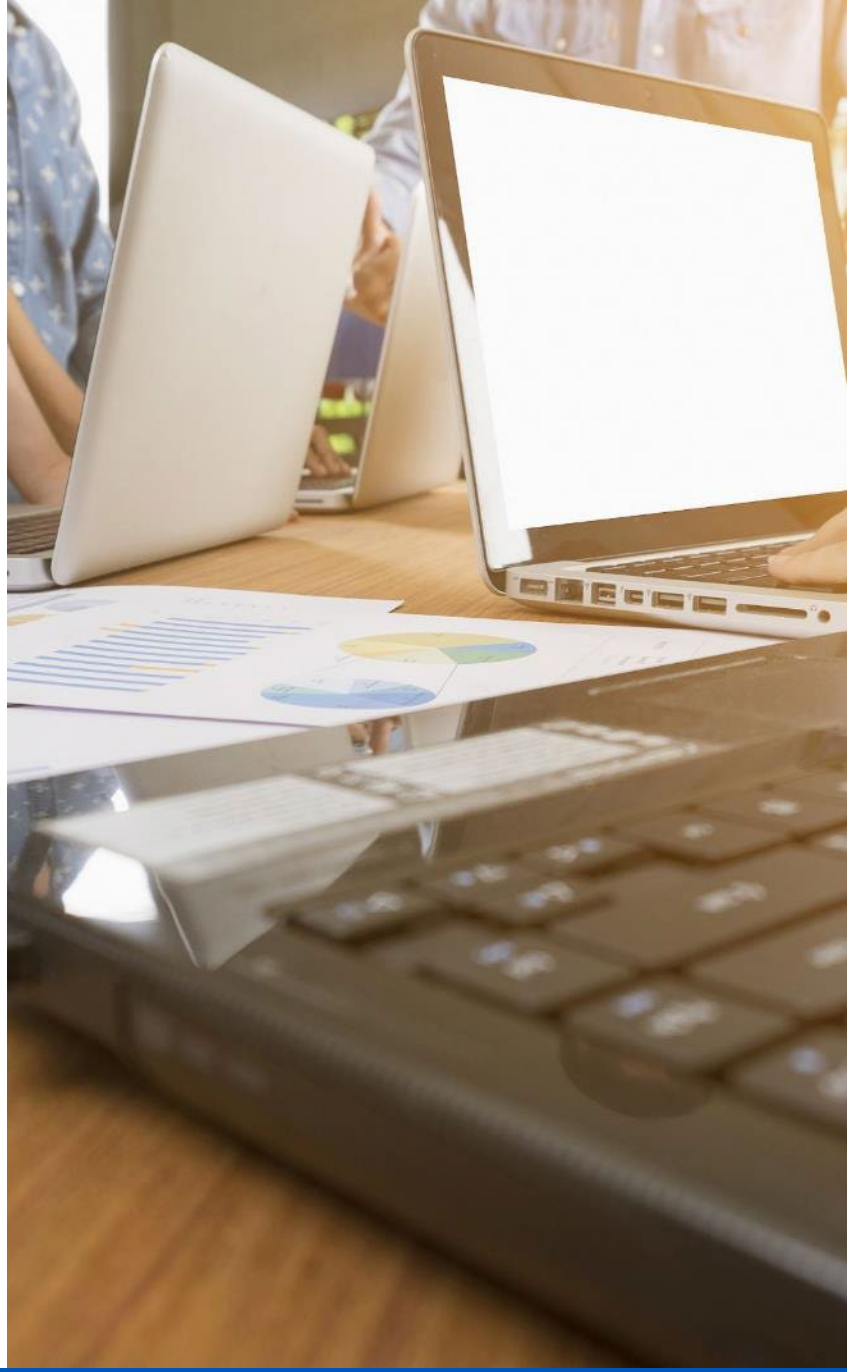
```
new_dict['language']
```

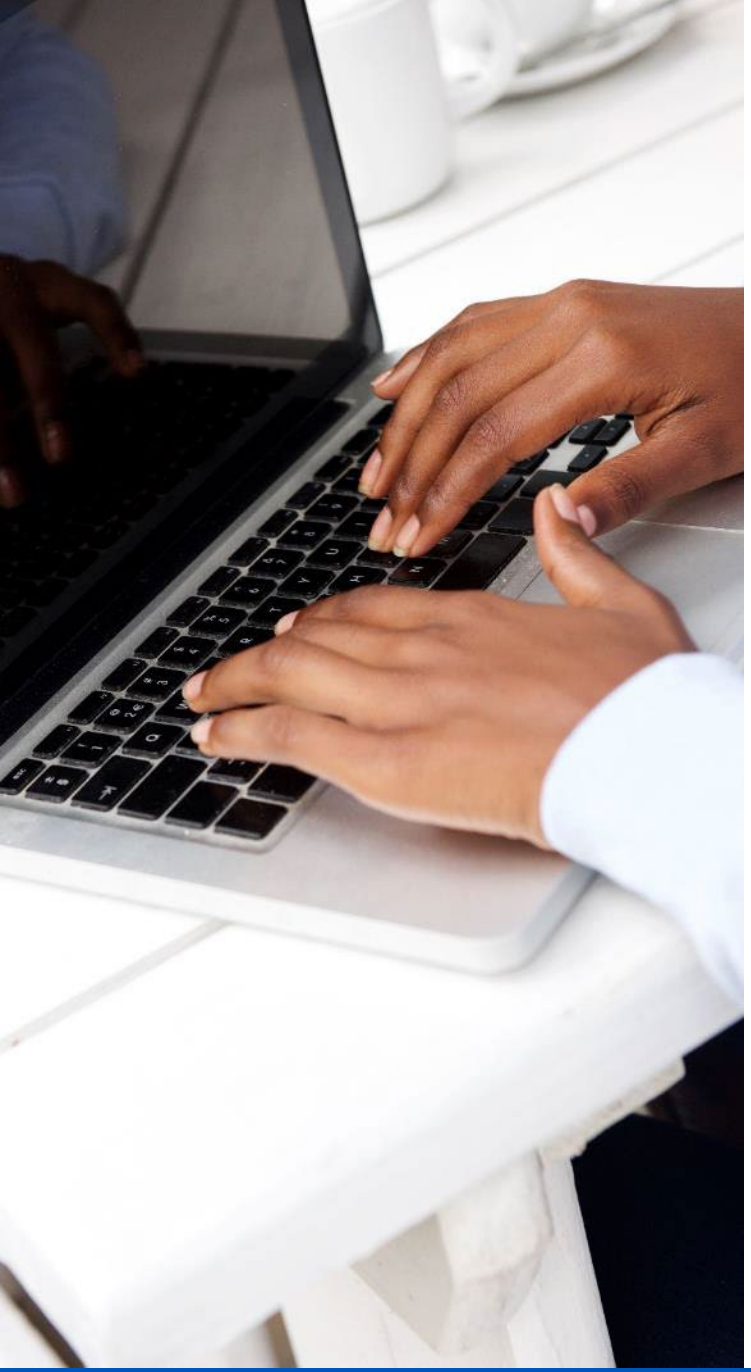
```
'python'
```

```
new_dict['database']
```

```
'mysql'
```

Contents



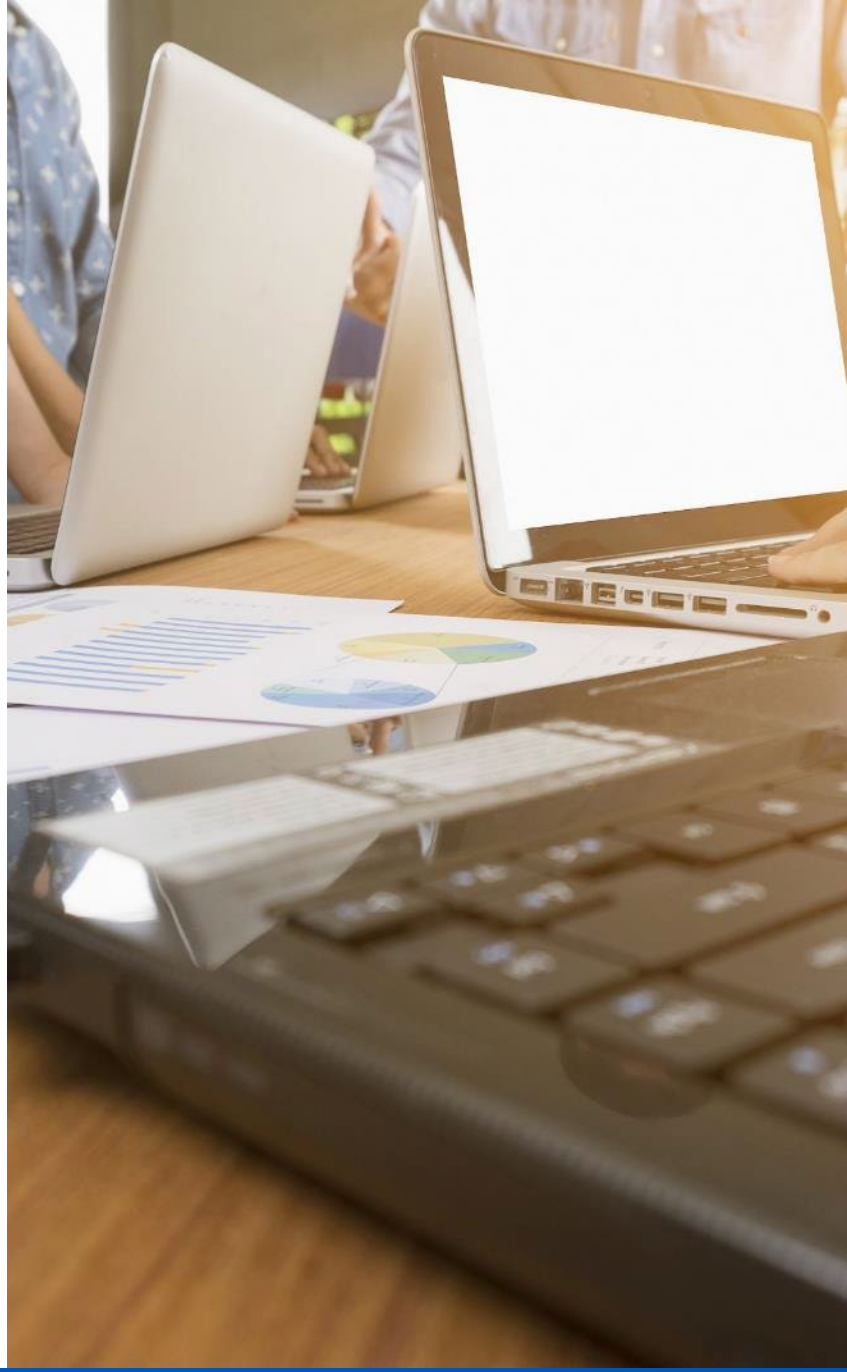


Hands-On Exercise 2.1

Jupyter Exercise Dashboard

In your Jupyter Exercise Dashboard, please refer to Hands-On Exercise 2.1: Working With Data in Python

Contents



Exploratory Data Analysis

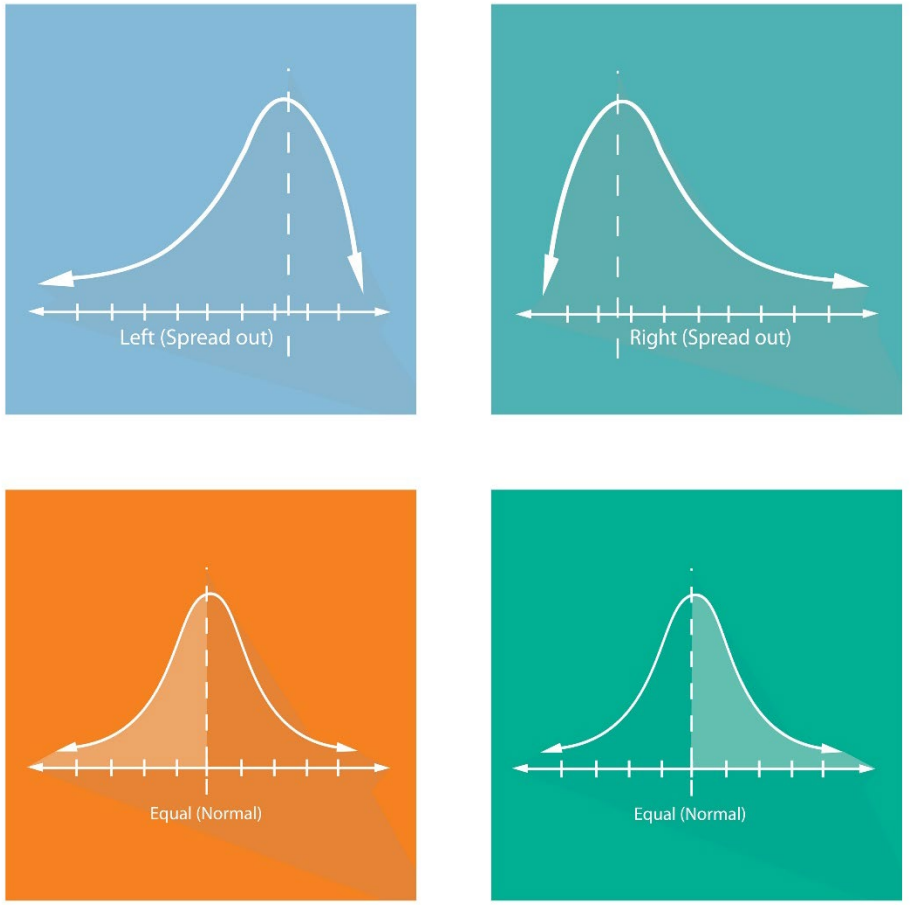
- **Exploratory Data Analysis (EDA) techniques are used to**
  - Summarize the data
    - These techniques can help in developing hypotheses about the real-world event that the data is capturing
  - Help find patterns in the data

Exploratory Data Analysis in Summary

- **Descriptive statistics help describe your data's distribution**
- **A measure of central tendency and dispersion are needed to describe your data's distribution statistically**
  - Ideally, your data fits the descriptions of a normal distribution with data distributed evenly on either side of the measure of central tendency
  - The following are measures of central tendency: mean, median, and mode
  - The following are measures of dispersion: range, variance, and standard deviation
- **Histograms and box plots can help you illustrate your data's distribution**
  - Your descriptive statistics, histograms, and/or box plots together help you describe the nature of your data

Exploratory Data Analysis in Summary

- **Descriptive statistics summarize a given dataset**
- **Include measures of central tendency and measures of variability (spread)**
- **Measures of central tendency include the mean, median, and mode**
- **Measures of variability include the standard deviation and variance**
  - May also include the minimum and maximum, as well as the kurtosis and skewness



Central Tendency

- **The following are three measurements of central tendency**
  - *Mean*: the sum of observed data points divided by the number of data records

```
iris['sepal_length'].mean()
```

```
5.843333333333335
```

  - *Median*: the middle data point (or average of the two middle data points if there is an even number of observations) when all data points are lined up in either ascending or descending order

```
iris['sepal_length'].median()
```

```
5.8
```

  - *Mode*: the most frequently occurring data point value in a dataset

```
iris['sepal_length'].mode()
```

```
0    5.0
dtype: float64
```

Measuring Dispersion

- **Dispersion refers to the spread of data about the data's center and can be measured by:**
  - *Variance*: a cumulative measure of individual data points' distance from the mean

```
iris['sepal_length'].var()

0.6856935123042505
```

  - *Standard deviation:* an average deviation of the data from the mean
    - Calculated as the square root of the variance:

```
iris['sepal_length'].std()
0.8280661279778629
```

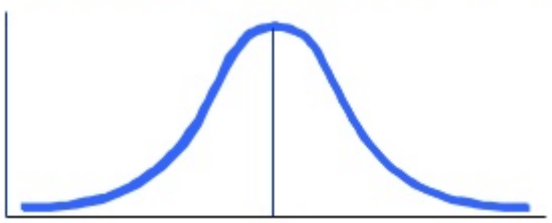
describe() Function

- The `describe` method displays a summary of some distribution measures

```
iris.describe()
```

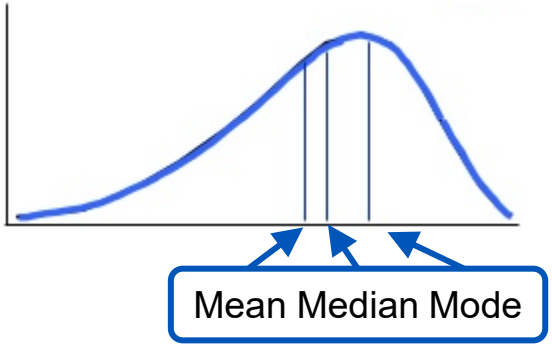
	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Skewness

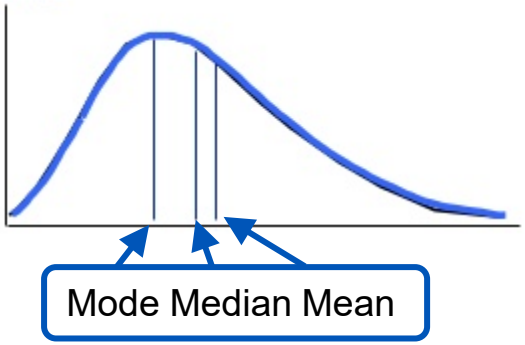


Mode = Mean = Median

Normal Distribution

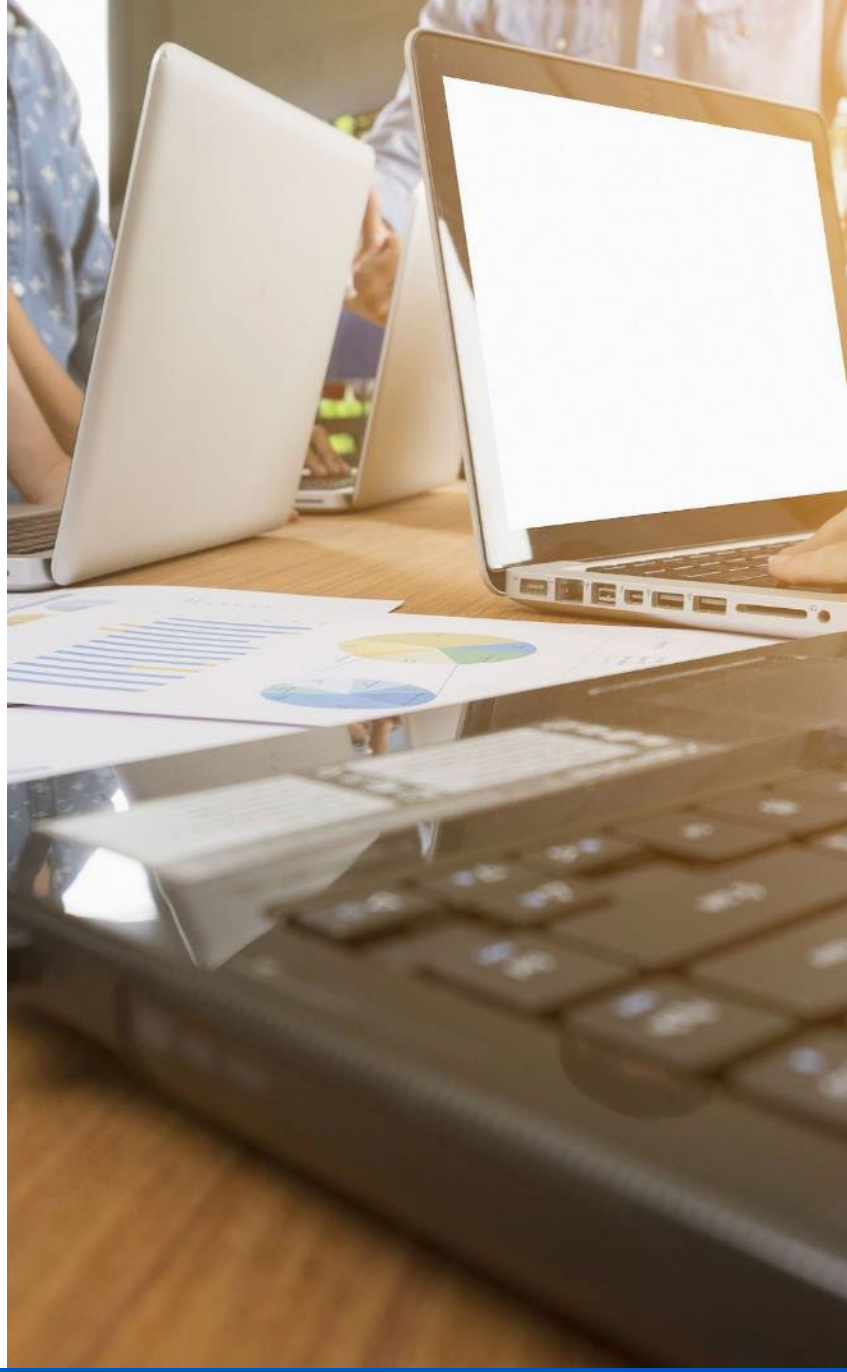


Skewed Left



Skewed Right

Contents



Visualizing Data

- **When interpreting or presenting findings on data distribution, it is helpful to provide a visual of the data's distribution in addition to the numbers**
- **For example, a histogram shows the frequency (count) of values in the dataset over a series of intervals that covers the range of the data**
  - A histogram is a great way of initially visualizing the data's distribution
  - Gives a sense of the central tendency and dispersion of data around that center before calculating any statistics
- **The `matplotlib` package is the most common package used for visualizations in Python**

pandas Plotting Methods

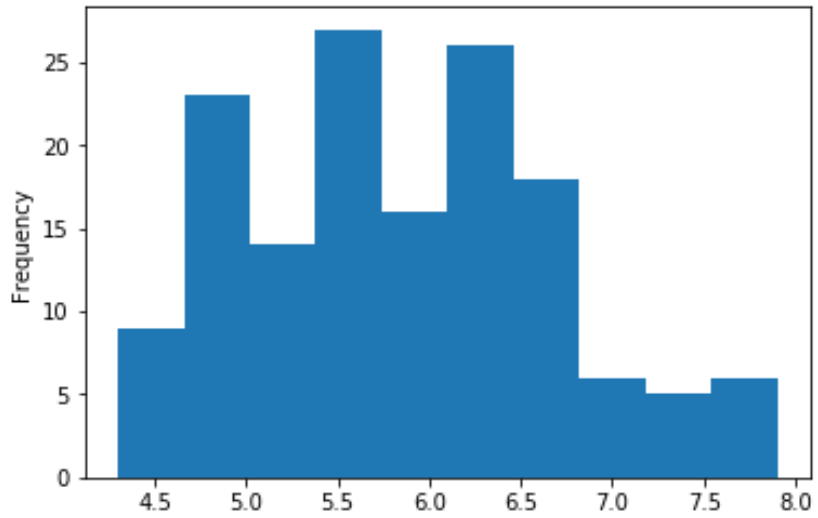
- **pandas plotting methods are convenient wrappers around matplotlib calls**
  - For example, the plot method on a Series or DataFrame is a simple wrapper around `plt.plot()`
  - Uses defaults and auto-formats in order to simplify plotting requirements
  - Will need to use `matplotlib.pyplot` directly to add figure labels and axis labels to your diagrams
- **Line plots are the default**
  - Other plot styles can be generated by specifying the type in the `kind` keyword argument to `plot()`:
    - `'bar'` or `'barh'` for bar plots
    - `'hist'` for histogram
    - `'box'` for boxplot
    - `'kde'` or `'density'` for density plots
    - `'area'` for area plots
    - `'scatter'` for scatter plots
    - `'pie'` for pie plots

Visualizing Numeric Data: Histograms

- **A histogram is a visual display of data using rectangles whose area is proportional to the frequency of a variable and whose width is equal to the class interval**
  - *Note: In Jupyter, the command sometimes needs to be executed a second time, before the plot will display*

```
iris['sepal_length'].plot(kind='hist')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x16cdfc18>
```

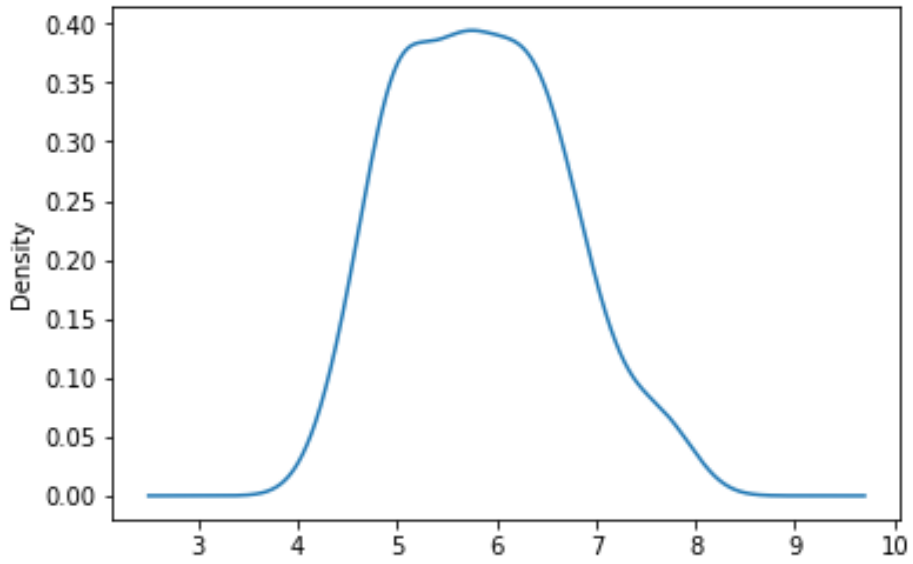


Visualizing Numeric Data: Density Plots

- **Kernel density plots are a much more effective way to view the distribution of a variable**

```
iris['sepal_length'].plot(kind='kde')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x101d5240>
```

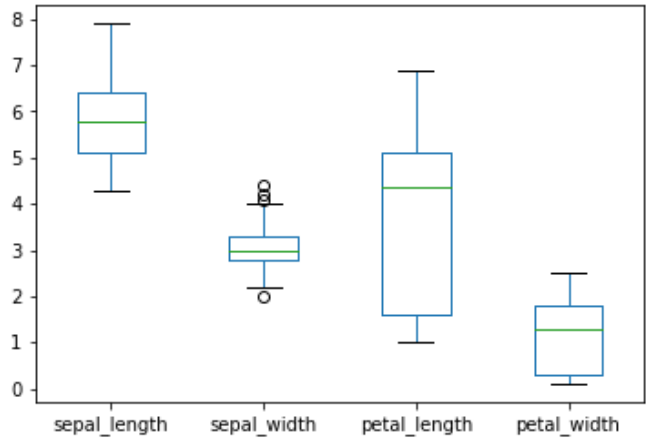


Visualizing Numeric Data: Box Plots

- **A box plot illustrates the distribution of the data**
  - Made up of: median, minimum value, maximum value, Quartile 1 value, and Quartile 3 value
  - Lower 25 percent (Quartile 1) of the data occurs between the bottom edge of the box and the bottom edge of the lower whisker
  - Upper 25 percent (Quartile 3) of the data occurs between the top edge of the box and the top edge of the upper whisker
- **To display a box plot:**

```
iris.plot(kind='box')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x16ca32b0>
```



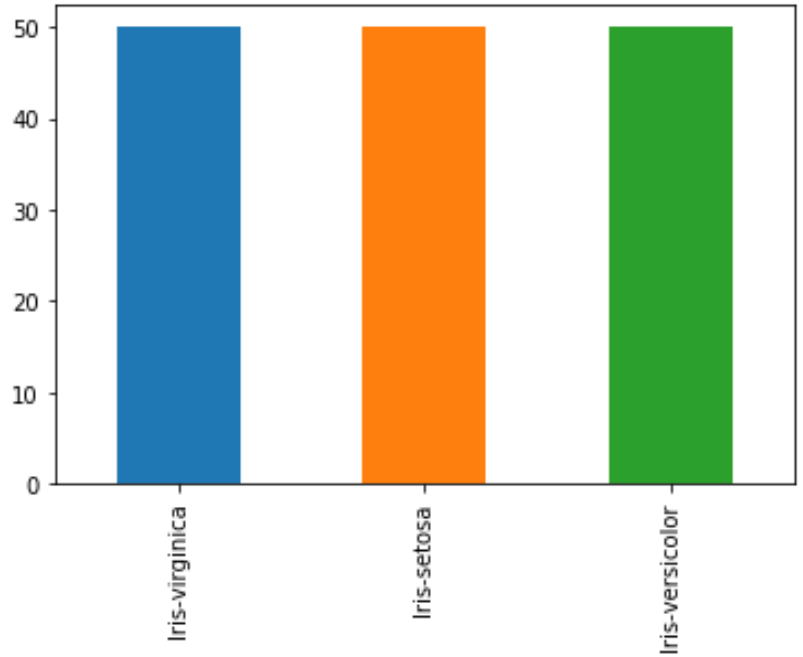
A side-by-side box plot can be used to visually compare the central tendencies of multiple datasets using the middle line of the box (representing the median value of the dataset)

Visualizing Categorical Data: Bar Charts

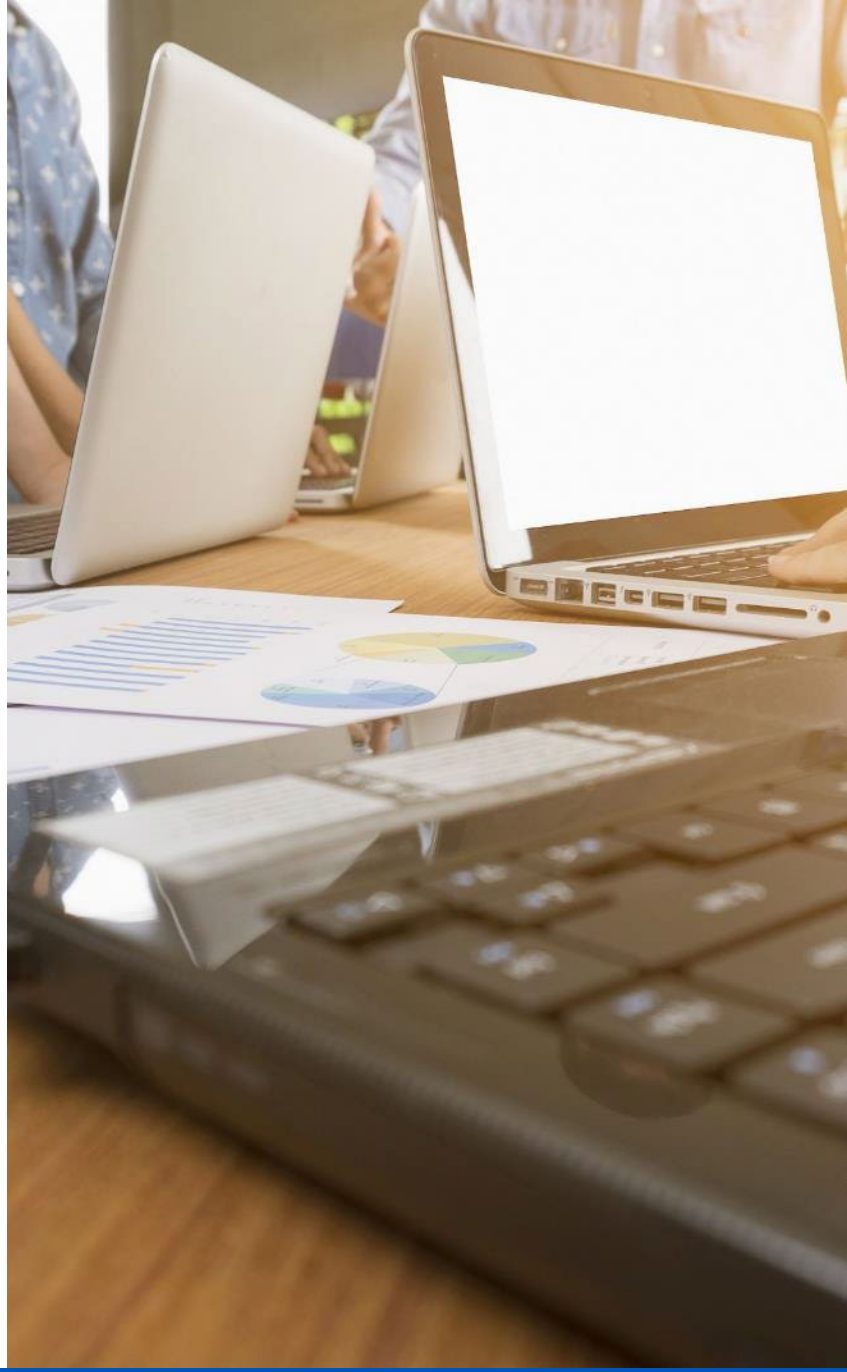
- **Bar Charts display counts of the number of observations using vertical bars for each categorical variable**

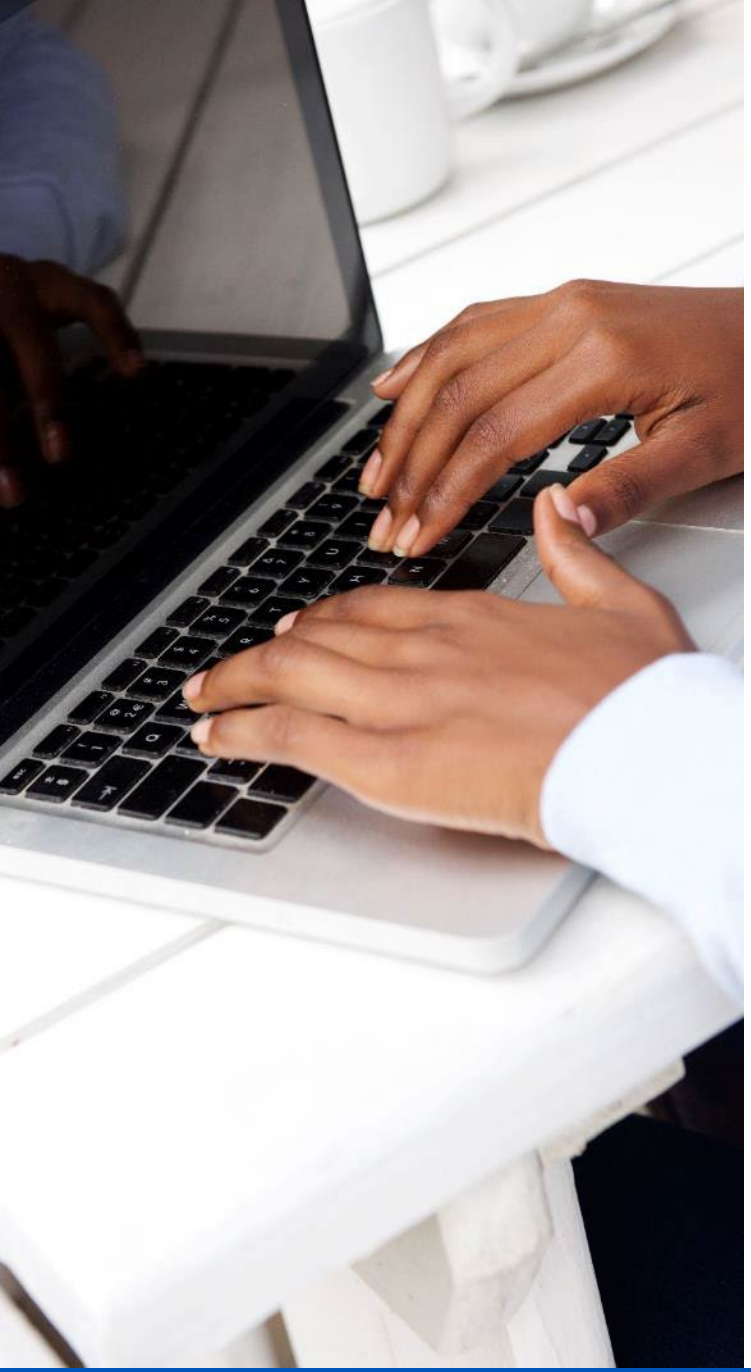
```
iris['species'].value_counts().plot(kind='bar')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x17893160>
```



Contents



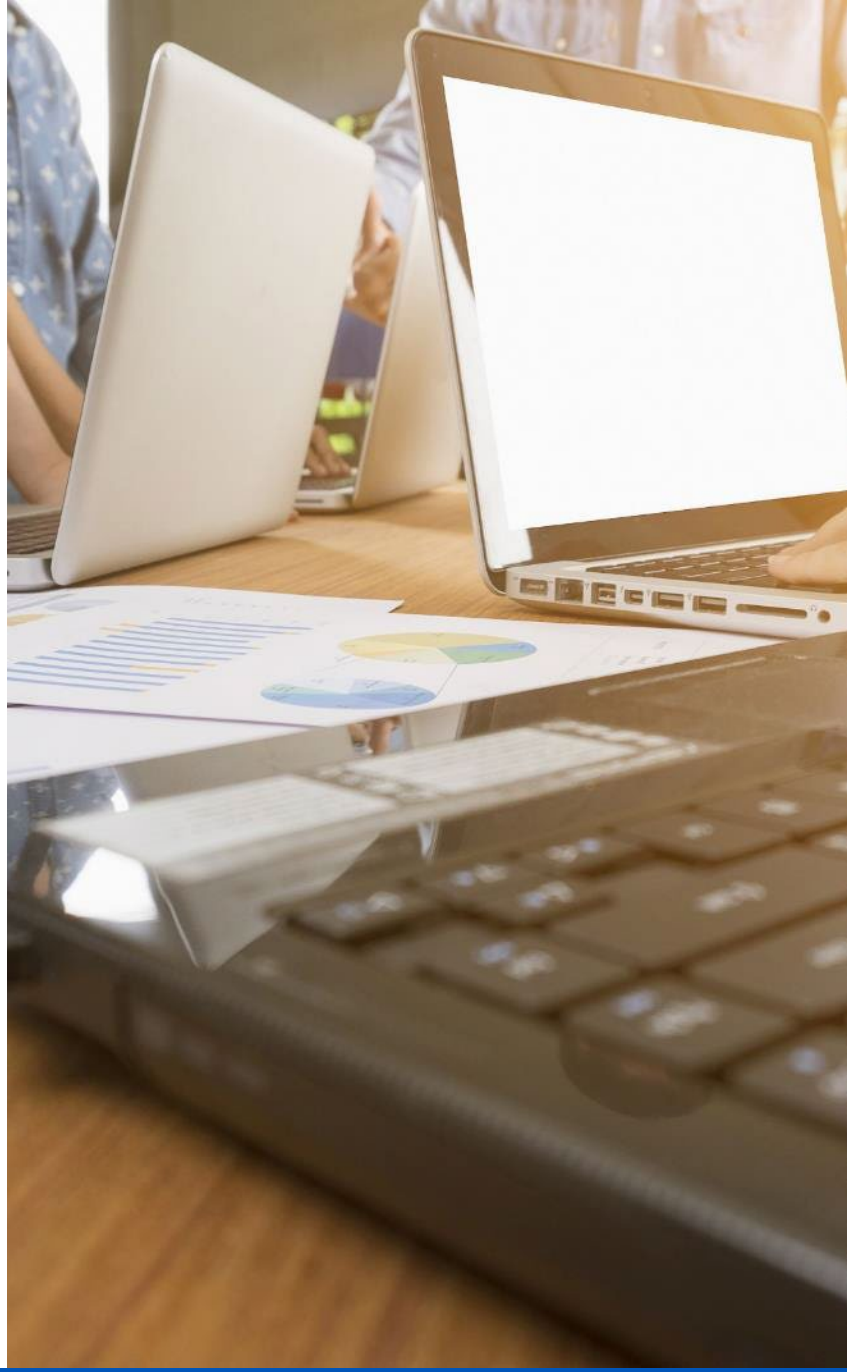


Hands-On Exercise 2.2

Jupyter Exercise Dashboard

In your Jupyter Exercise Dashboard, please refer to Hands-On Exercise 2.2: Exploring and Visualizing Data in Python

Contents



Cleaning and Transforming the Data

- **Raw data typically needs to be cleaned and transformed in different ways, in order to model it**
  - This is particularly true when dealing with unstructured data
- **Raw data may come in many varied formats**
  - Machine-generated log files
  - Excel spreadsheets from the HR department
  - JSON data retrieved from the LinkedIn Application Programming Interface (API)
  - Downloaded academic papers
- **Regardless of its raw form, data needs to be cleaned and transformed so that it is more or less normalized (similar to relational databases)**
  - Variables are in separate columns
  - Observations are in separate rows

Cleaning and Transforming the Data

- **Typical transformation tasks are**
  - Rescaling the data
    - In order for variables to be compared, they need to be on a similar scale
  - Dealing with missing values
    - Replacing with mean, median, mode, constants, or special symbols
  - Remapping
    - Allows various types of binning, log transformation, etc.
- **It is advisable to script required data changes as evidence of results**

Rescaling

- **Normalization**
  - Rescaling numeric attributes into the range 0 and 1

```
from sklearn import preprocessing

preprocessing.normalize(iris.iloc[:,[0,1,2,3]])
```

```
Out[44]: array([[0.80377277, 0.55160877, 0.22064351, 0.0315205 ],
                [0.82813287, 0.50702013, 0.23660939, 0.03380134],
                [0.80533308, 0.54831188, 0.2227517 , 0.03426949],
                [0.80003025, 0.53915082, 0.26087943, 0.03478392],
                [0.790965  , 0.5694948 , 0.2214702 , 0.0316386 ],
                [0.78417499, 0.5663486 , 0.2468699 , 0.05808704],
                [0.78010936, 0.57660257, 0.23742459, 0.0508767 ],
                [0.80218492, 0.54548574, 0.24065548, 0.0320874 ],
                [0.80642366, 0.5315065 , 0.25658935, 0.03665562],
                [0.81803119, 0.51752994, 0.25041771, 0.01669451],
                [0.80373519, 0.55070744, 0.22325977, 0.02976797],
                [0.786991  , 0.55745196, 0.26233033, 0.03279129],
                [0.82307218, 0.51442011, 0.24006272, 0.01714734],
                [0.8025126 , 0.55989251, 0.20529392, 0.01866308],
                [0.81120865, 0.55945424, 0.16783627, 0.02797271],
                [0.77381111, 0.59732787, 0.2036345 , 0.05430253],
                [0.79428944, 0.57365349, 0.19121783, 0.05883625],
                [0.80327412, 0.55126656, 0.22050662, 0.04725142],
                [0.8068282 , 0.53788547, 0.24063297, 0.04246464],
```

Missing Values

- **Missing values need to be treated with care, as they can affect or skew analysis if not treated correctly**
- **Missing data in Python appears as `NaN`**
  - `NaN` is not a string or a numeric value

```
titanic = pd.read_csv('titanic.csv')
titanic['Age'][25:30]
```

```
25    38.0
26     NaN
27    19.0
28     NaN
29     NaN
Name: Age, dtype: float64
```

Missing Values

- Use `.isnull()` to detect missing values

```
titanic['Age'][25:30].isnull()
```

```
25    False
26     True
27    False
28     True
29     True
Name: Age, dtype: bool
```

Imputation of Missing Values

- Allows replacement of missing data with substitute values
- Zero/mean/median/mode or a constant can be imputed for missing values
- The `fillna()` method can "fill in" NA values with non-null data
- Example:

```
titanic['Age'][25:30]
```

```
25    38.0
26     NaN
27    19.0
28     NaN
29     NaN
Name: Age, dtype: float64
```

```
titanic['Age'][25:30].fillna(0)
```

```
25    38.0
26     0.0
27    19.0
28     0.0
29     0.0
Name: Age, dtype: float64
```

- Other "fill" methods can be used in a similar way:
  - `ffillna()`
  - `bfillna()`
  - `mean()`
  - `dropna()`

Remapping/Recode Operations

- **Categorical data may be encoded as follows:**

```
titanic.insert(5, 'GenderCode', pd.Categorical(titanic['Sex']).codes)
```

```
titanic[['Sex', 'GenderCode']]
```

	Sex	GenderCode
0	male	1
1	female	0
2	female	0
3	female	0
4	male	1
5	male	1

pandas Methods

- **The pandas libraries have many functions and methods that can be used for data transformation**
  - Some additional pandas methods we will use in this course:

Conversion	
`.as_type(dtype)` `.to_numeric()`	`.to_datetime()` `.to_timestamp()`
Numeric	
`.round()` `.div()`	`.diff()`
String	
`.str.upper()` `.str.lower()`	`.str.replace()` `.str.format()`

- **For an extensive list of pandas functions and methods:**
  - https://pandas.pydata.org/pandas-docs/stable/api.html

Additional Packages: NumPy and SciPy

- **The package NumPy is typically used in Python data analytics for efficient numerical computation**
  - Provides support for large multi-dimensional arrays and matrices
  - Contains a vast library of mathematical functions to perform operations on these arrays

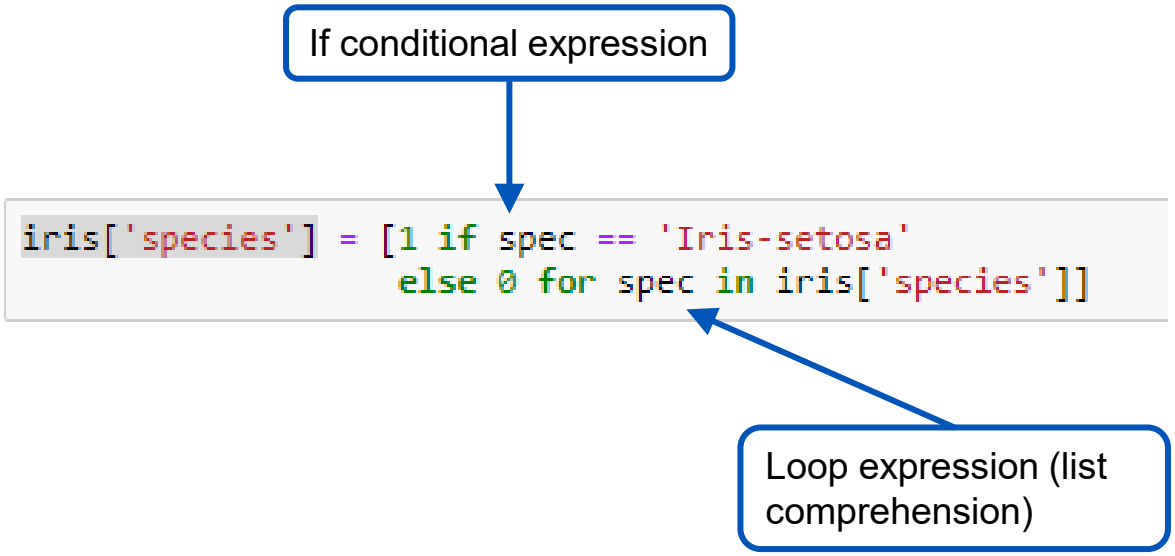
    https://numpy.org/doc/stable/reference/index.html

- **The SciPy package builds on NumPy's functionality, offering a broader range of advanced mathematical, scientific, and engineering algorithms and tools for more complex computations and data analysis**

  https://docs.scipy.org/doc/scipy/reference/

Looping Over Elements in a Column: List Comprehensions

- **List comprehensions can also be used to carry out replacements**



Data Preparation: Data Type Conversion

- **Data types of the `DataFrame` columns may be converted `.astype()`**

```
a = [['x', '1.5', '6.3'], ['y', '20', '0.15'], ['z', '8','0.4']]

df = pd.DataFrame(a, columns = ['A', 'B', 'C'])

df
```

	A	B	C
0	x	1.5	6.3
1	y	20	0.15
2	z	8	0.4

```
df.dtypes
```

```
A    object
B    object
C    object
dtype: object
```

Converts columns B and C to a float datatype

```
df[['B','C']] = df[['B','C']].astype(float)
df.dtypes
```

```
A     object
B    float64
C    float64
dtype: object
```

Grouping Rows By a Shared Value

- **The `groupby()` operation must always be paired with the function (i.e., `mean()`, `sum()`, `count()`, etc.) that is to be applied to the resulting group**
  - Otherwise summary info only will be displayed

```
iris.groupby('species').count()
```

species	sepal_length	sepal_width	petal_length	petal_width
Iris-setosa	50	50	50	50
Iris-versicolor	50	50	50	50
Iris-virginica	50	50	50	50

Data Preparation: Matrix Manipulation—concat()

- You will frequently need to combine datasets in different ways
- The `concat()` function combines `DataFrames` by rows

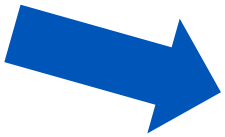
```
concat([dataframeA, dataframeB])
```

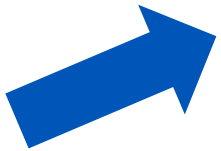
```
df1
```

	Subtype	Gender	Expression
0	A	M	-0.54
1	A	F	-0.80
2	B	M	-1.03
3	C	M	-0.41

```
df2
```

	Subtype	Gender	Expression
0	D	F	2.50
1	D	F	-0.15
2	D	M	-0.20
3	D	F	-0.04





```
pd.concat([df1,df2])
```

	Subtype	Gender	Expression
0	A	M	-0.54
1	A	F	-0.80
2	B	M	-1.03
3	C	M	-0.41
0	D	F	2.50
1	D	F	-0.15
2	D	M	-0.20
3	D	F	-0.04

Merging DataFrames

- **The `merge()` function joins two `DataFrames` by common column(s)**
  - Can also be called as a method (e.g., `df1.merge(df2)`)

	CourseId	Course	CategoryId
0	101	Intro to Oracle	10
1	102	Intro to SqlServer	10
2	201	Hadoop Programming	30
3	301	Intro to Java	40

Courses

	CategoryId	Category
0	10	RDBMS
1	20	Systems
2	30	Big Data
3	40	Java

Categories

```
pd.merge(Courses, Categories, on='CategoryId', how='inner')
```

	CourseId	Course	CategoryId	Category
0	101	Intro to Oracle	10	RDBMS
1	102	Intro to SqlServer	10	RDBMS
2	201	Hadoop Programming	30	Big Data
3	301	Intro to Java	40	Java

Functions

- **User-defined functions can also be easily added in Python**
- **To master some of the more advanced analytics examples in this course, you need a solid foundation in how Python functions work**
- **The structure of a function is:**

```
def functionname( parameters ):
    executable code
    return [expression]
```

  - Example:

```
def firstfunction():
    str = 'Hello World'
    return str

firstfunction()
```

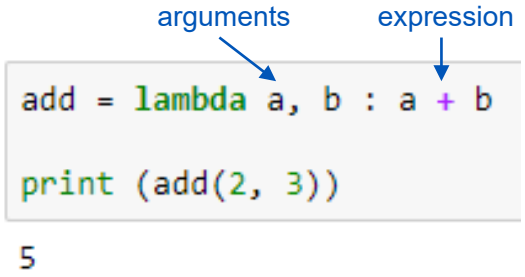
```
'Hello World'
```

Lambda Functions

- **Used for creating one-time anonymous functions**
  - Allow creation of a function when it is needed only and then applied to the relevant data set.
- **Syntax:**

```
lambda arguments : expression
```

- **Example:**



- Can have any number of arguments but only one expression
- Cannot contain any statements
- Returns a function object
- Often used as a parameter to a method such as `.apply()`

Looping Over Elements in a Column: The `.apply()` Method

- **Used to execute either a Python built-in function or a custom-defined function across every element of a dataset**
- **What if the Fare price paid on the Titanic had been $1 more than they were?**
  - In this simplistic example, the lambda function takes as input the Fare price from the `DataFrame`, adds 1 to it, returning the new price

```
titanic = pd.read_csv('titanic.csv')
titanic['Fare'].head()
```

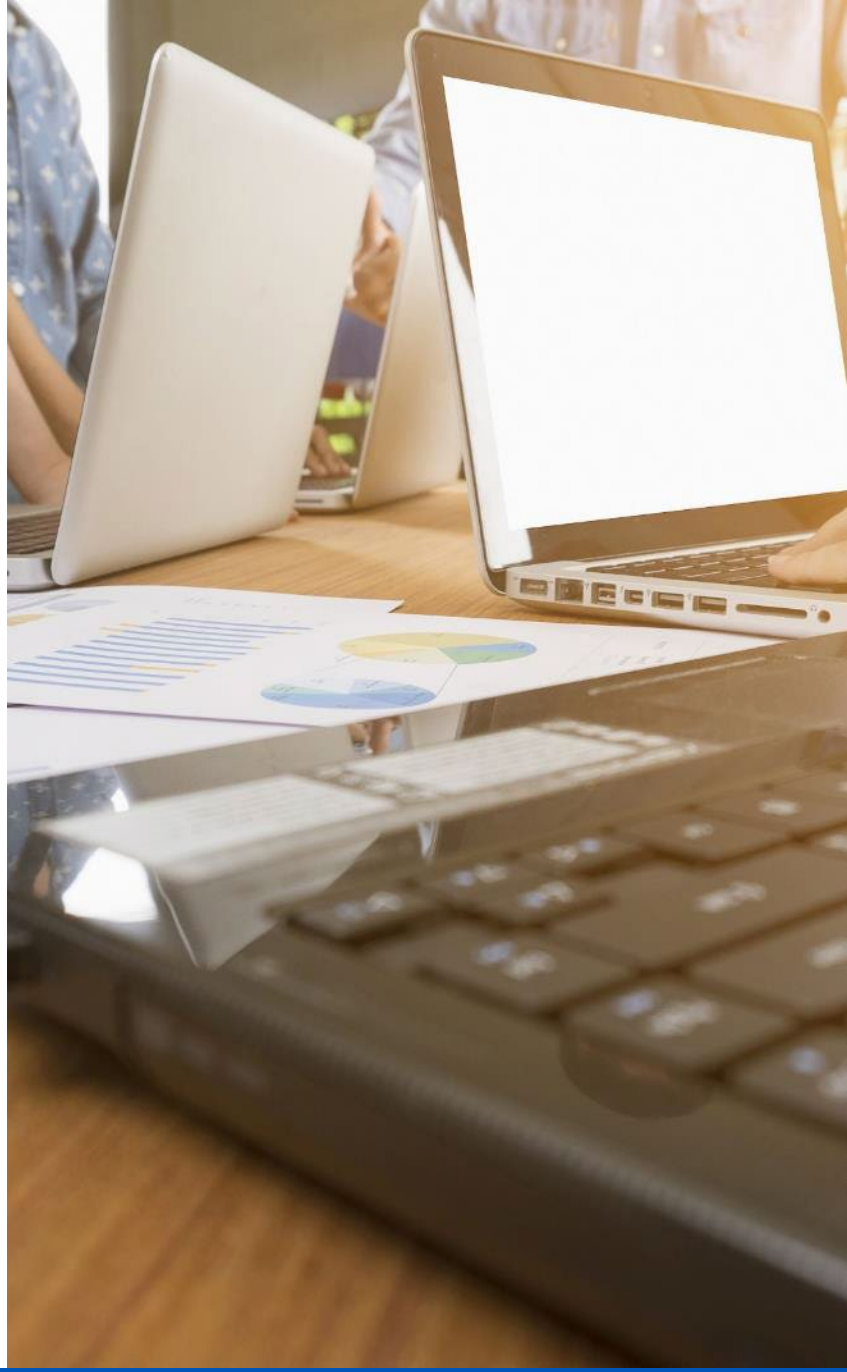
```
0     7.2500
1    71.2833
2     7.9250
3    53.1000
4     8.0500
Name: Fare, dtype: float64
```

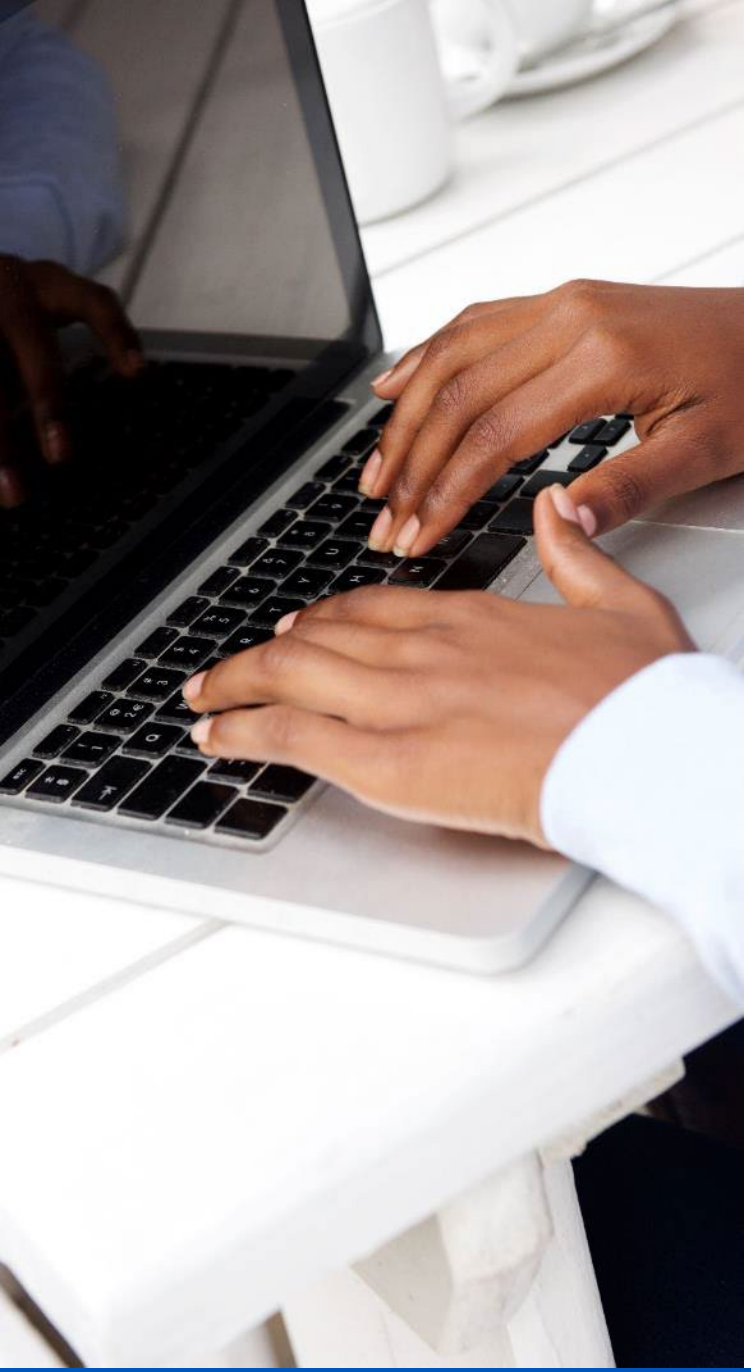
```
titanic['Fare'].apply(lambda price: 1 + price).head()
```

```
0     8.2500
1    72.2833
2     8.9250
3    54.1000
4     9.0500
Name: Fare, dtype: float64
```

Contents

Hands-On Exercise 2.3





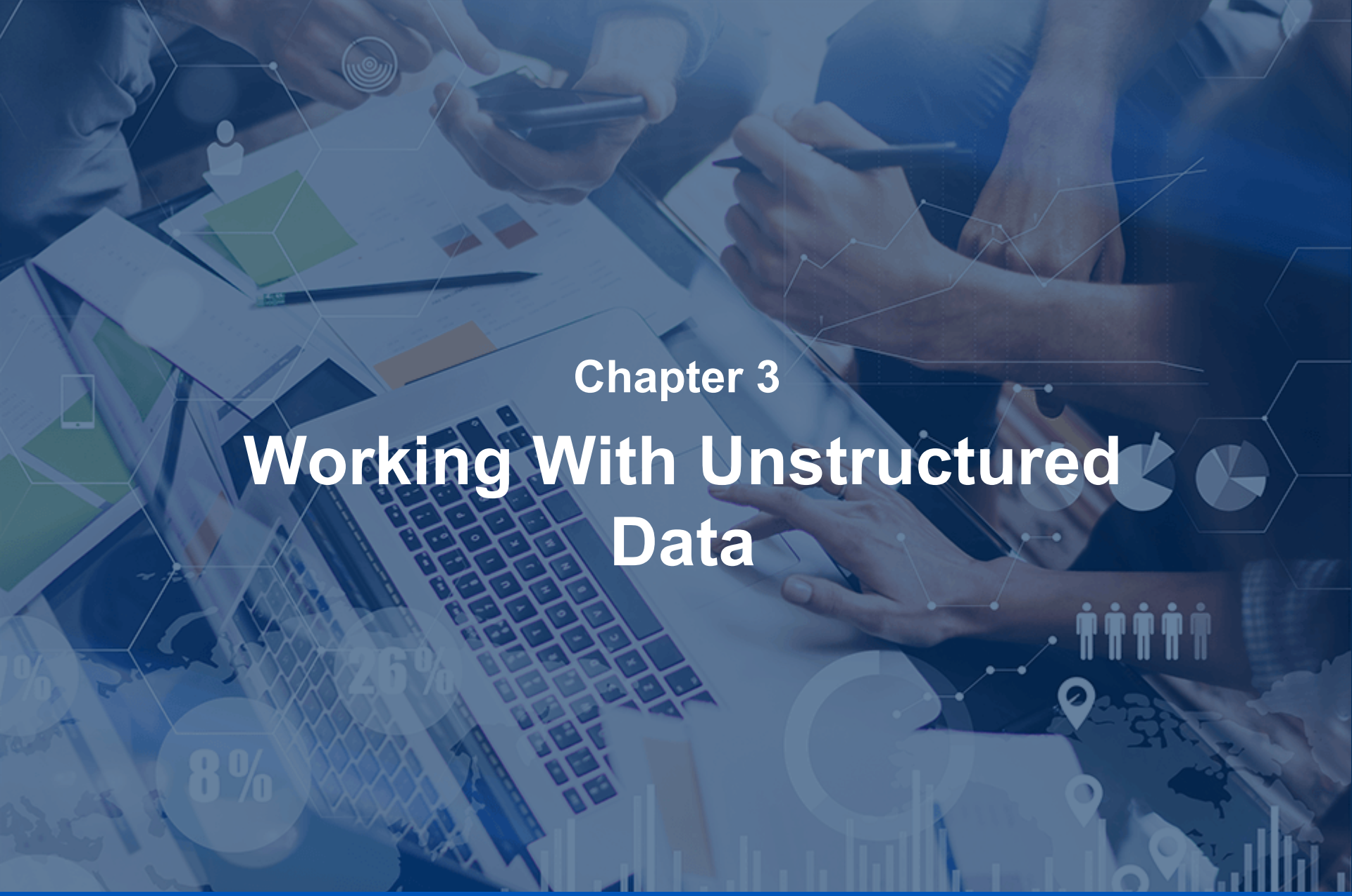
Hands-On Exercise 2.3

Jupyter Exercise Dashboard

In your Jupyter Exercise Dashboard, please refer to Hands-On Exercise 2.3: Transforming Data in Python in Preparation for Analysis

Objectives

- Explore data sets with Python
- Employ data visualization as an aid to understanding data sets
- Prepare data for analysis



Chapter 3

Working With Unstructured Data

LEARNING TREE INTERNATIONAL

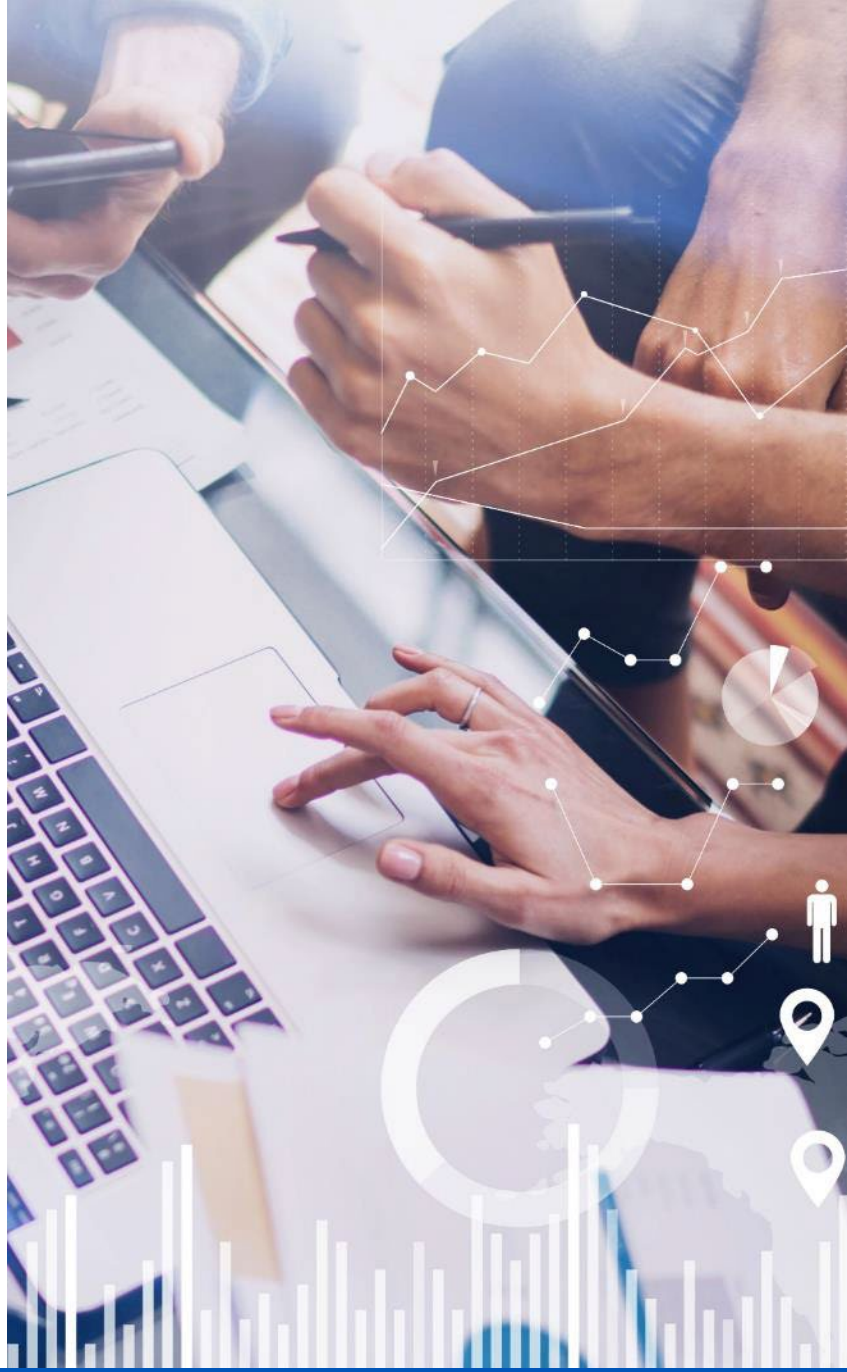
Objectives

- Review how unstructured data is used in decision-making
- Preprocess unstructured data in preparation for text mining
- Explore the concept of stemming
- Investigate the meaning of stop words
- Prepare a term-document matrix (TDM) of unstructured documents in preparation for analysis

Contents

Introduction to Unstructured Data

- Natural Language Processing (NLP)
- Feature Engineering in NLP
- Hands-On Exercise 3.1



Unstructured Data Applications in AI

- **With the growth of unstructured data generation across various sources, many new applications have arisen in recent years**
- **Use cases include:**
  - Computer vision applications
    - E.g., image recognition, object detection, facial recognition
  - Audio Recognition applications
    - E.g., voice assistants, voice search
  - Natural language processing (NLP) applications
    - E.g., chatbots, sentiment analysis, machine translation
- **Research into novel neural network architectures for generative AI has added to this growth**

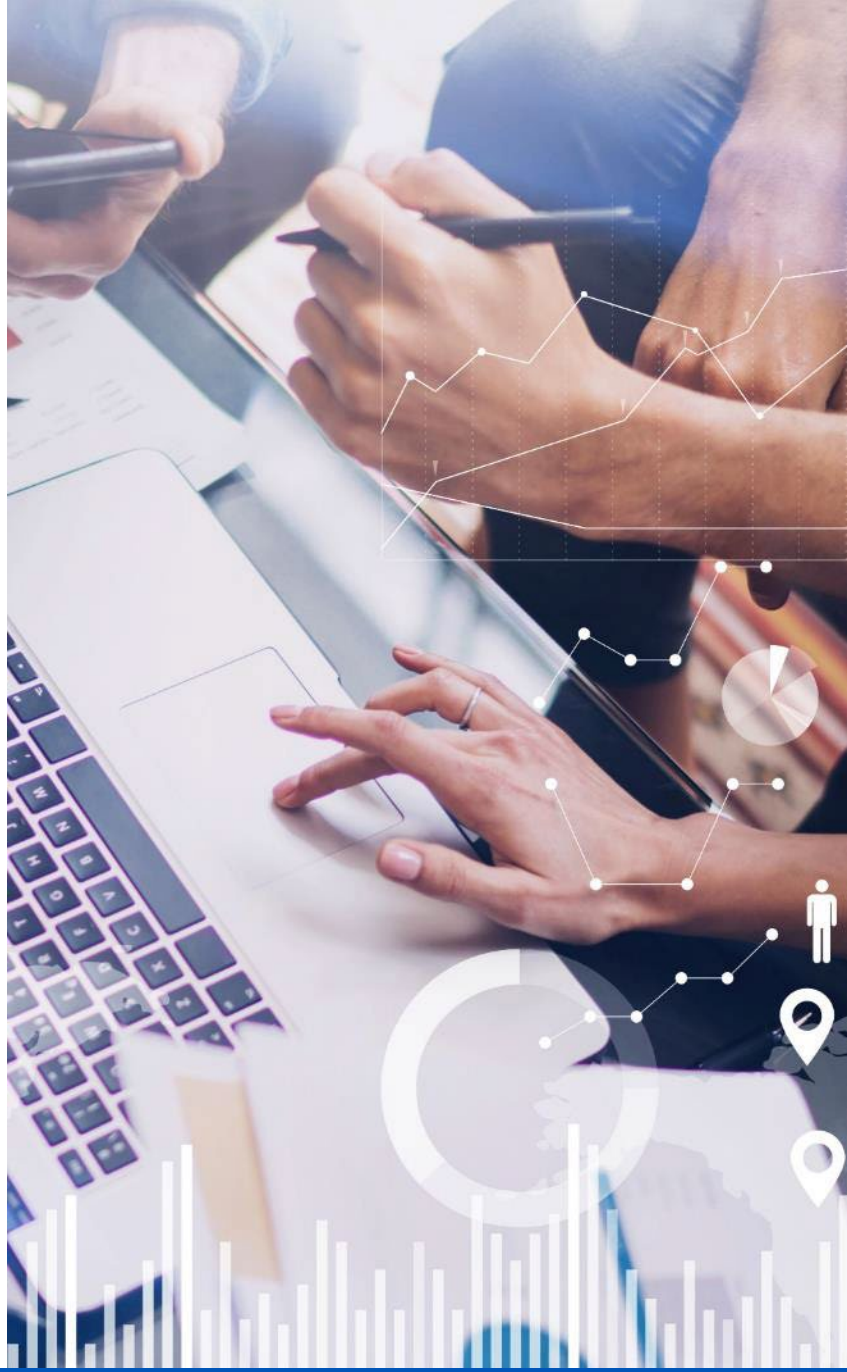
Exploring Computer Vision

- **A number of image processing techniques are typically employed as a preliminary step in computer vision tasks. These techniques may include:**
  - Filtering, segmentation, and feature extraction
- **Computer vision machine learning can then be used to interpret and make decisions based on visual data**
  - Examples: identifying objects in images, recognizing faces, automating quality control in manufacturing, self-driving cars, security systems, medical imaging

Exploring Audio Recognition

- **Similarly signal processing techniques such as the following are typically involved in audio recognition:**
  - Feature extraction, noise reduction
- **Machine learning can then be used to interpret and make decisions based on this data**
  - Examples: voice assistants, smart speakers, automated transcription, security monitoring

Contents



Natural Language Processing (NLP)

- **NLP refers to a broad set of techniques for analysis and manipulation of natural language**
- **It has applications in many different fields that involve:**
  - Understanding text (customer service, social media analysis, sentiment analysis, named entity recognition)
    and/or
  - Generating text (content creation, chatbots, machine translation, text summarization)

Approaches to NLP

A variety of techniques are typically used to work with natural language data

1. Preprocessing:

- Tokenization: splitting text into words, sentences, or characters
- Normalization: converting text to a consistent format, like lowercase or removing punctuation to ensure consistency in interprets
- Stemming and lemmatization: reducing words to their base form (e.g., "running" becomes "run")
- Stop word removal: removing common words (e.g., "the," "a")

2. Feature engineering:

- TF-IDF (term frequency-inverse document frequency): assigning weights to words based on their frequency within a document and rarity across the entire dataset. This helps identify keywords relevant to specific documents.
- N-grams: analyzing sequences of words (bigrams, trigrams) to capture phrases or recurring word combinations
- Part-of-speech (POS) tagging: identifying the grammatical function of each word (noun, verb, adjective, etc.) to help understand the structure of the text

Approaches to NLP

3. Word embeddings:

- Represents words as numerical vectors
  - Captures semantic relationships between words
    - Words with similar meanings will have closer vectors in this high-dimensional space
- Popular word embedding techniques include Word2Vec and GloVe

4. Deep learning techniques:

- Recurrent neural networks (RNNs): these models can process sequential data like text by considering the context of previous words
- Transformers: a “self-attention” neural network architecture that excels at understanding relationships between words in a sentence, even if distant

► **The specific techniques used will depend on the application and the type of analysis being performed**

- E.g., sentiment analysis might focus on techniques like TF-IDF and word embeddings to understand the emotional tone of the text, while topic modeling might leverage techniques like N-grams to identify recurring themes within a document collection

Large Language Models (LLMs)

- **LLMs are powerful, generalized AI models trained on massive amounts of textual data using deep learning techniques**
  - They learn complex relationships between words, phrases, and entire sentences
  - Can be built using different neural network architectures (usually transformers along with other techniques) to achieve their goal of understanding and generating human language
  - Enables:
    - Language translation
    - Varied content creation
    - Question answering in an informative way
- **Represents a significant advancement in natural language processing (NLP)**
- **Can perform a wide range of tasks with minimal task-specific fine-tuning**

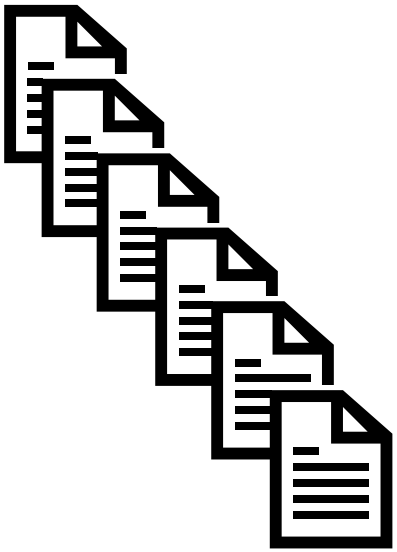
LLMs vs. Traditional NLP approaches

- **Both LLMs and traditional approaches to NLP are complementary to each other, with the best choice depending on specific needs and priorities**

	Natural Language Processing (NLP)	Large Language Models (LLMs)
Focus	Focuses on specific, well-defined tasks like text classification, sentiment analysis, or named entity recognition	LLMs broader understanding and generation of human language means they can handle a wider range of tasks eg. poems, code, email, social media content, etc., but may not achieve the same level of accuracy on specific tasks
Data Requirements and Training	Involves smaller, curated datasets with faster training times	Require massive amounts of unlabeled text data, computational resources and time
Adaptability	May struggle to adapt to new situations or unseen data since they are trained for specific tasks	Can be more adaptable due to their ability to learn general language patterns
Interpretability	More interpretable as the logic behind the model's decisions can be traced back to the rules or algorithms used	Challenging to interpret due to complexity of neural networks. It can be difficult to understand why an LLM generates a specific output.

Pre-processing Unstructured Data

- **The aim in preprocessing unstructured data is to represent the document/video/image, etc., in terms of its measurable features**
  - This will typically be a vector or matrix structure



Documents

The word *tomorrow* appears 1 time in Document 4

	Docu1	Docu2	Docu3	Docu4	
tomorrow				1	
analytics		1	1		
learn	1				
historical	1				
…					

Term-document matrix

How to Prepare Unstructured Text Data

- **Before the data can be vectorized, textual data typically needs to be preprocessed**
- **Preparing unstructured data for ML typically includes:**
  1. Cleaning and feature extraction
     - Lower casing
     - Removing punctuations, HTML tags, special characters, alphanumerics
     - Removing most frequent or rare words
     - Correcting spelling mistakes
     - Removal of stop words
     - Stemming or lemmatization
  2. Tokenization and encoding the text data into integers values that can be used by machine learning algorithms

Reading in the Data File

- **To open a file, use the `open()` function**
  - Requires two arguments
    - The file path or file name and the mode it should be opened in
      - "r" (default): read only, "w": if the file exists then content is deleted and opened it to write, and "a": append
- **To read the contents of the file, use the `read()` function**

```
In [1]: with open('shakes.txt') as shakes_file:
   ...:     shakes = shakes_file.read()
   ...:

In [2]: shakes[:500]
Out[2]: 'The Project Gutenberg EBook of The Complete Works of William
Shakespeare, by\nWilliam Shakespeare\n\nThis eBook is for the use of
anyone anywhere at no cost and with\nalmost no restrictions
whatsoever.  You may copy it, give it away or\nre-use it under the
terms of the Project Gutenberg License included\nwith this eBook or
online at www.gutenberg.org\n\n** This is a COPYRIGHTED Project
Gutenberg eBook, Details Below **\n**     Please follow the copyright
guidelines in this file.     **\n\nTitle: The Comple'
```

Converting Text to Lowercase

- Use the `lower()` method to return a copy of the string in which all case-based characters have been lowercased

```
shakes.lower()

the project gutenberg ebook of the complete works
of william shakespeare, by\nwilliam shakespeare\n\nthis ebook is
for the use of anyone anywhere at no cost and with\nalmost no
restrictions whatsoever.  you may copy it, give it away or\nre-
use it under the terms of the project gutenberg license
included\nwith this ebook or online at www.gutenberg.org\n\n**
this is a copyrighted project gutenberg ebook, details below
**\n**     please follow the copyright guidelines in this file.
**\n\ntitle: the comple'2]:
```

Removing Numbers and Punctuation Marks

- **To remove numbers and punctuation marks, a built-in package known as `re` may be used**

```
import re

shakes
'te Works of William Shakespeare\n\nAuthor: William
Shakespeare\n\nPosting Date: September 1, 2011 [EBook
#100]\nRelease Date: January, 1994\n\nLanguage:
English\n\nCharacter set encoding: ASCII\n\n*** START OF THIS
PROJECT GUTENBERG EBOOK COMPLETE WORKS--WILLIAM SHAKESPEARE
***\n\n\n\n\nProduced by World Library, Inc., from their Library
of the Future\n\n\n\n\nThis is the 100th Etext file presented by
Project Gutenberg, and\nis presented in cooperation with World
Library, Inc., from their\nLibrary of the Future and Sh'

re.sub(    "[^a-zA-Z]", " ", shakes )
'te Works of William Shakespeare  Author  William
Shakespeare  Posting Date  September          EBook
Release Date  January        Language  English  Character set
encoding  ASCII      START OF THIS PROJECT GUTENBERG EBOOK
COMPLETE WORKS  WILLIAM SHAKESPEARE         Produced by World
Library  Inc   from their Library of the Future     This is the
th Etext file presented by Project Gutenberg  and is presented
in cooperation with World Library  Inc   from their Library of
the Future and Sh'
```

Replace anything that is *not* a lowercase letter (a-z) or an uppercase letter (A-Z), and replace it with a space

[ ] indicates group membership
^ indicates “not”

Tokenization

- **The corpus* is then normally vectorized through a process called *tokenization***
  - The text is split into tokens
    - A token may refer to a word, punctuation mark, number, or alphanumeric
  - Tokenization may result in loss of meaning
  - In English, white space or punctuation marks are often used to tokenize
    - More difficult in languages such as Chinese or Arabic, as there are no explicit boundaries

```
nltk.tokenize.word_tokenize(shakes)
['the',
 'project',
 'gutenberg',
 'ebook',
 'of',
 'the',
 'complete',
 'works',
 'of',
 'william',
 'shakespeare',
 'by',
 'william',
 'shakespeare',
```

*A corpus is a body of text containing a large number of sentences.

Stop Word Removal

- **Dimensionality of the data**
  - Each token represents a dimension
  - Some tokens are more important than others in determining meaning
  - Removal of stop words acts as a technique for reducing the dimensionality
- ***Stop words* refers to the common words that do not contribute as highly to meaning; they are**
  - Articles—a, an, the
  - Conjunctions—and, or…
  - Prepositions—as, by, of…
  - Pronouns—you, she, he, it…
- **Other non–context-related words can also be removed**

Removing Stop Words With NLTK

- The `nltk` library greatly simplifies the task of stop-word identification and removal
- Load the `nltk` library for text processing

```
import nltk
```

- To download text data sets including stop words

```
nltk.download('stopwords')
```

NLTK = Natural Language Toolkit

Import and View Stop Words

- **Import and print a list of English language stop words**

```
from nltk.corpus import stopwords

stopwords.words("english")
```

```
['i',
 'me',
 'my',
 'myself',
 'we',
 'our',
 'ours',
 'ourselves',
 'you',
 'your',
 'yours',
 'yourself',
 'yourselves',
 'he',
 'him',
 'his',
 'himself',
 'she',
```

Remove Stop Words

- **To remove stop words from text**

```
sentence = "the quick brown fox jumped over the lazy dogs"

[i for i in sentence.split() if i not in stopwords.words('english')]
['quick', 'brown', 'fox', 'jumped', 'lazy', 'dogs']
```

Stemming

- **Stemming is the process for reducing derived words to their stem or root form**
  - Typically achieved by removing -ing, -s, -er, -ed, etc.
  - For example: “playing,” “player,” “plays,” “ played”
  - Stemmed word “play”
- **NLTK provides several famous stemmers interfaces, such as the Porter Stemmer, the Lancaster Stemmer, and the Snowball Stemmer**

Stemmers

- **The Porter Stemmer algorithm follows a set of five generic rules to generate stems**
  - Often generate stems that are not actual English words
  - Simple and fast
  - For example, the word, “destabilized”, is stemmed to “dest”
- **Lancaster Stemmer has many rules indexed by the last letter of a suffix**
  - A rule specifies either a deletion or replacement of an ending
  - No stemming occurs if a word starts with a vowel and has only two letters left, or if a word starts with a consonant and has only three characters left
    - Simple but not as fast
  - Over-stemming may cause stems to have no meaning
  - For example, the word, “destabilized”, is stemmed to “destabl”
- **Snowball Stemmers can be used to create non-English stemmers, or one’s own language stemmer**
  - Danish, Dutch, English, French, German, Hungarian, Italian, Norwegian, Portuguese, Romanian, Russian, Spanish, Swedish

Lemmatization

- **Lemmatization reduces derived words, ensuring that the root word belongs to the language**
  - Use where it is necessary to retrieve valid words, and speed is not as important
- **A root form is not produced where a word is given without context**
  - This is given as a parts-of-speech (POS) parameter

Stemming

- **Use NLTK to do a Porter Stemmer**

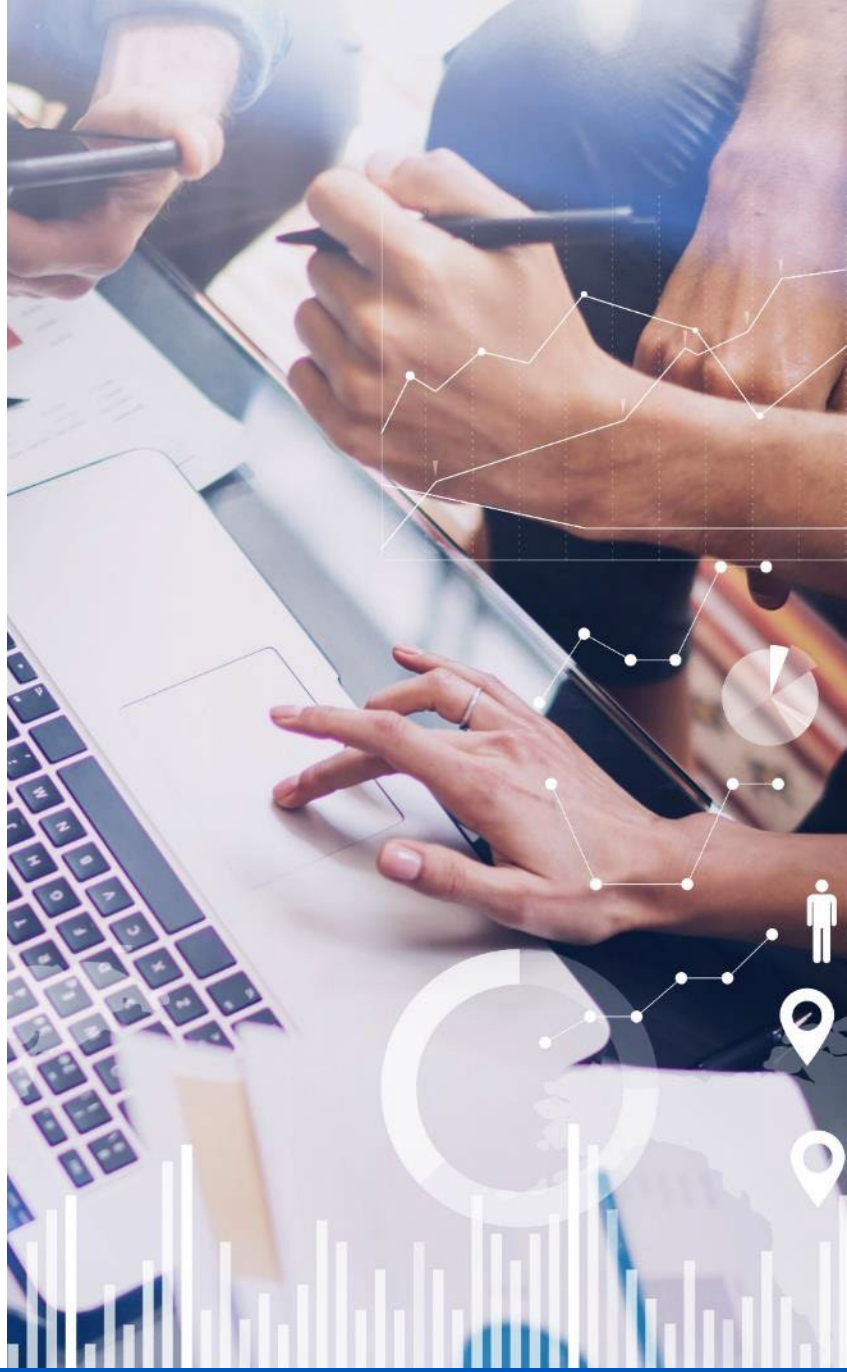
```
from nltk.stem import PorterStemmer

stemmer=PorterStemmer()

stemmed = []

sentence = "running jumping played presumably multiply


for item in sentence.split():
    stemmed.append(stemmer.stem(item))


stemmed
['run', 'jump', 'play', 'presum', 'multipli', 'owe']
```

Contents

- Introduction to Unstructured Data
- Natural Language Processing (NLP)

Feature Engineering in NLP

- Hands-On Exercise 3.1

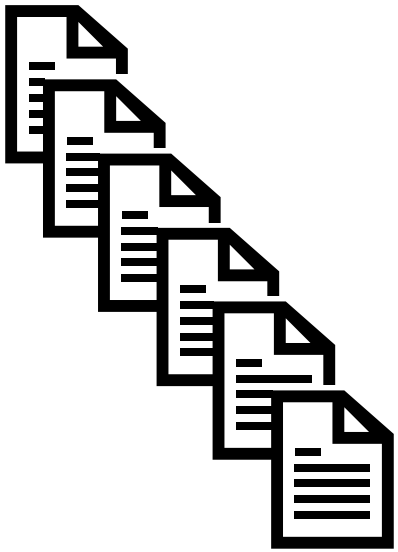


Term Frequency-Inverse Document Frequency

- **Some words are more important than others in determining meaning**
  - Each text word represents a dimension
- **To reduce the number of dimensions, TF-IDF is often used to prepare text for ML algorithms**
  - Text is processed into weighted vectors
- **Term frequency (TF)**
  - Number of times a term occurs in a document
  - Used to find out the relevance of a word in a document
    - The more frequent a word is, the more relevance the word holds in the context
- **Document frequency (DF)**
  - Number of documents that contain each word
  - Inverse document frequency (IDF) = log (Total No. of docs/DF)
  - TF-IDF = TF * IDF

Term Frequency—Inverse Document Frequency

- **The assumption behind TF-IDF is that terms that appear more frequently in a document should be given a higher weight, unless they also appear in a lot of documents**
- **To prepare the corpus for analysis, a term document matrix (TDM) is created**
  - Shows the terms and frequency of each word in the corpus



Documents

The word *tomorrow* appears 1 time in Document 4

	Docu1	Docu2	Docu3	Docu4	
tomorrow				1	
analytics		1	1		
learn	1				
historical	1				
…					

Term-document matrix

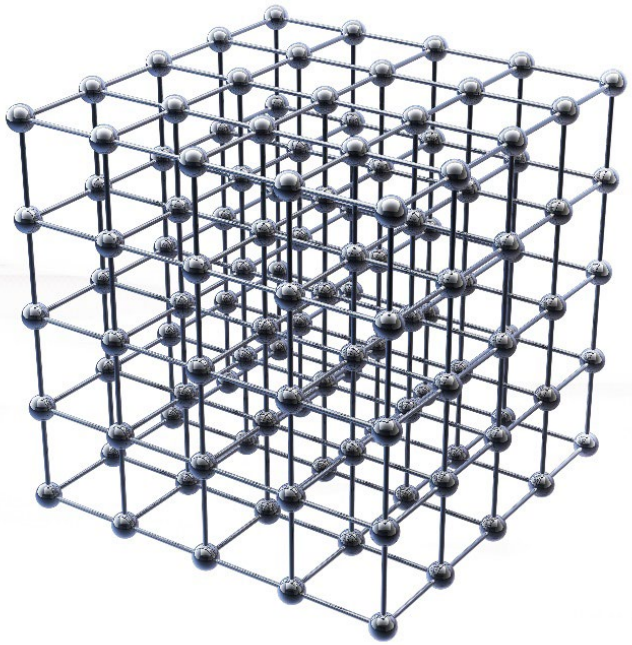
Building a TDM

- **To create the TDM, a number of functions from the `Scikit-learn` package are used**
- **The `TfidfVectorizer()` function is made up of a:**
  - `CountVectorizer()` function: Converts documents to a sparse matrix of token counts, and a
  - `TfidfTransformer()` function: Transforms a count matrix to a normalized *tf-idf* representation
- **The `fit_transform()` function learns the vocabulary and inverse document frequencies, and returns a TDM**

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(tokenizer=tokenize, stop_words='english')

tfs = tfidf.fit_transform(token_dict.values())
```

Exploring the TDM

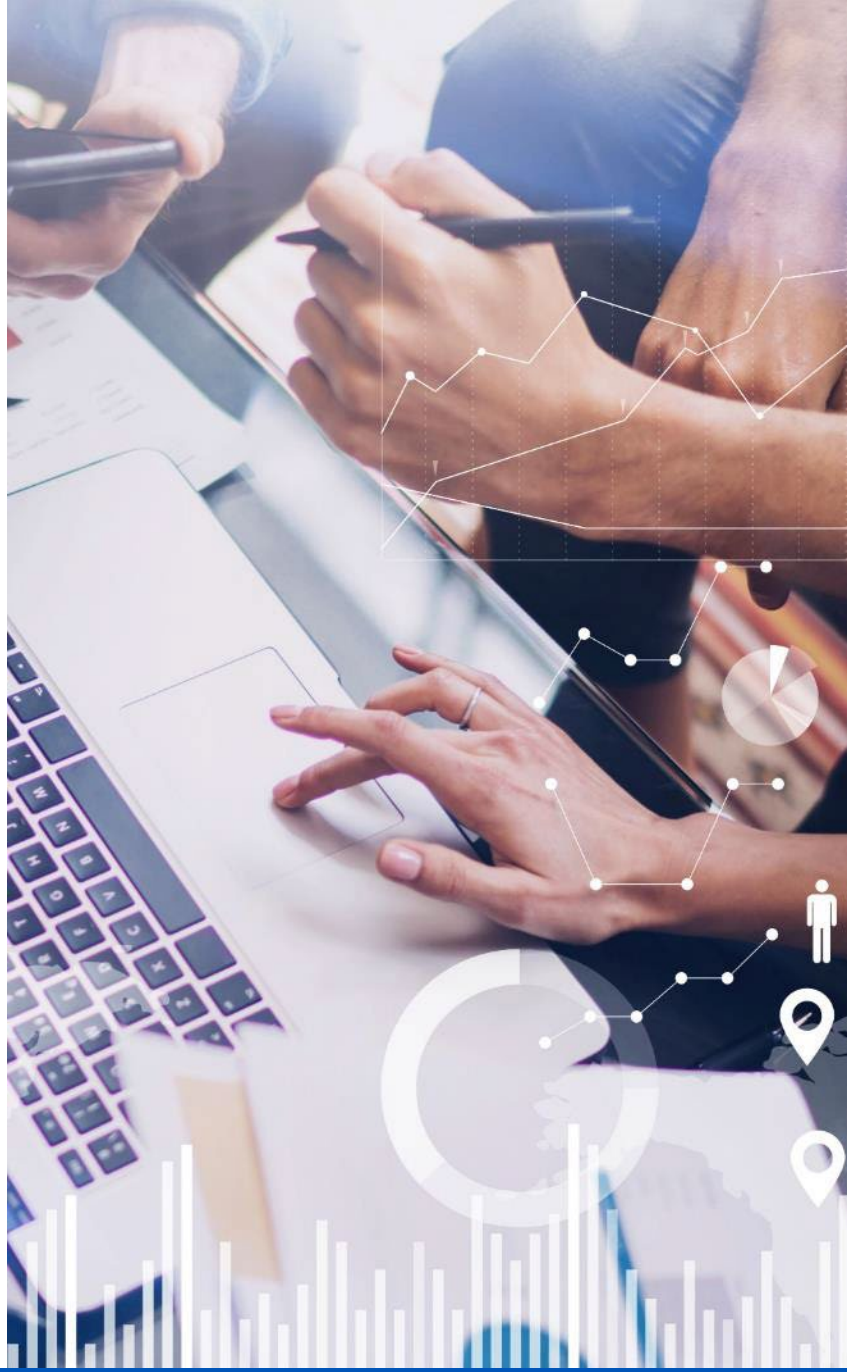
- **The `get_feature_names()` method can be used to see the words/features**
- **The `idf_` attribute can be used to view the inverse document frequencies**
- **The creation of the TDM is a pre-processing step when using unstructured textual data in Machine Learning models**
  - Example: a Classification model for detecting Spam email

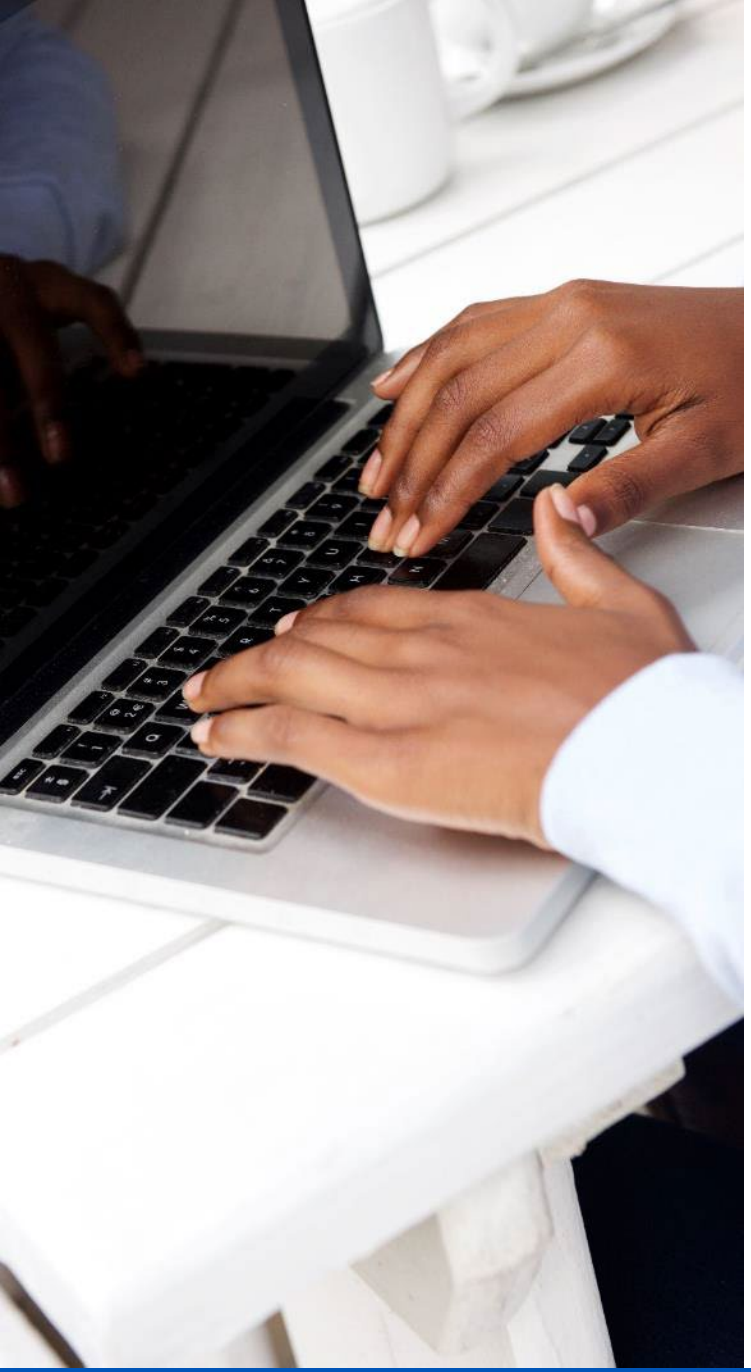


Contents

- Introduction to Unstructured Data
- Natural Language Processing (NLP)
- Feature Engineering in NLP

Hands-On Exercise 3.1





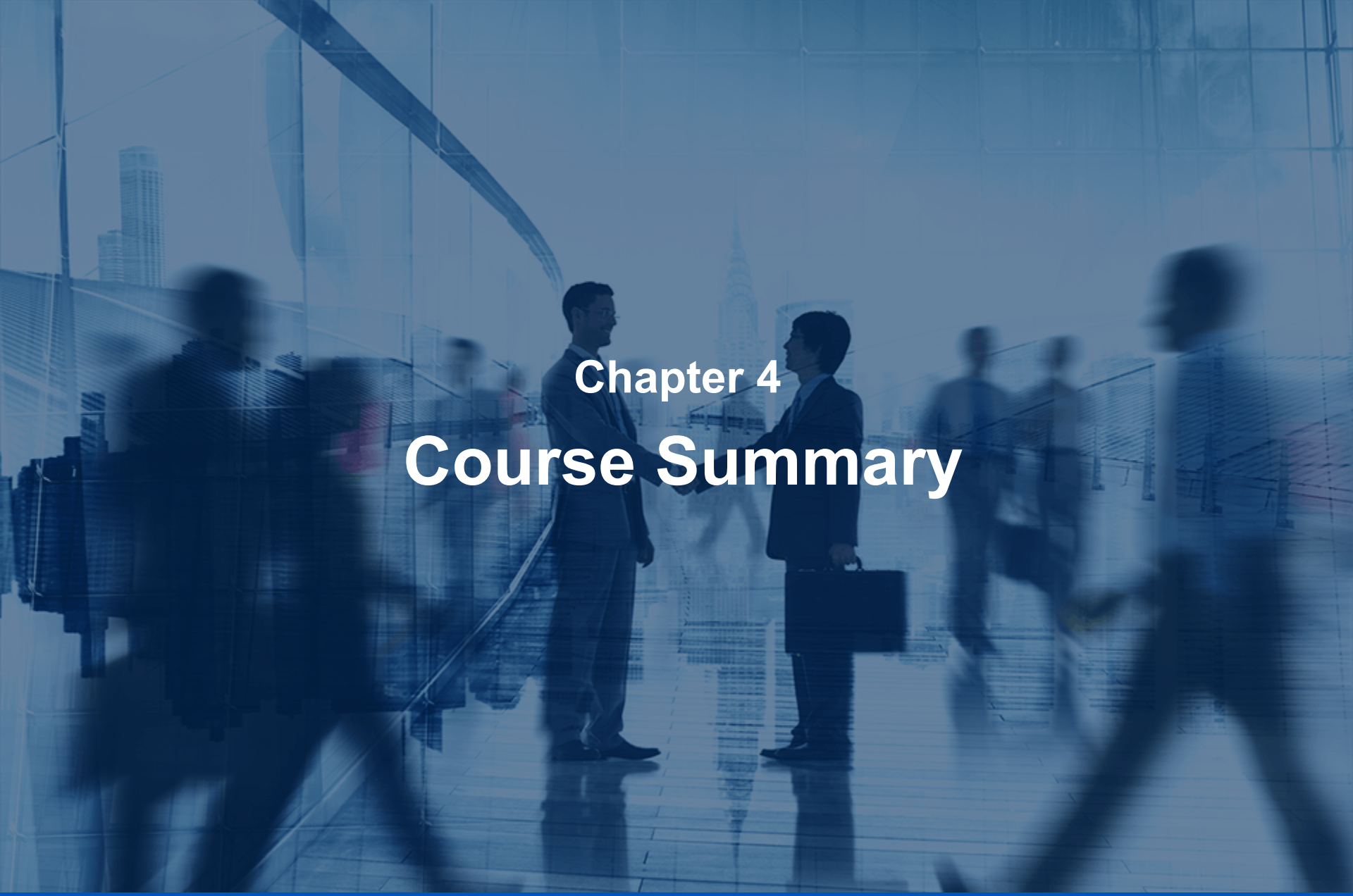
Hands-On Exercise 3.1

Jupyter Exercise Dashboard

In your Jupyter Exercise Dashboard, please refer to Hands-On Exercise 3.1: Text Mining

Objectives

- Review how unstructured data is used in decision-making
- Preprocess unstructured data in preparation for text mining
- Explore the concept of stemming
- Investigate the meaning of stop words
- Prepare a term-document matrix (TDM) of unstructured documents in preparation for analysis



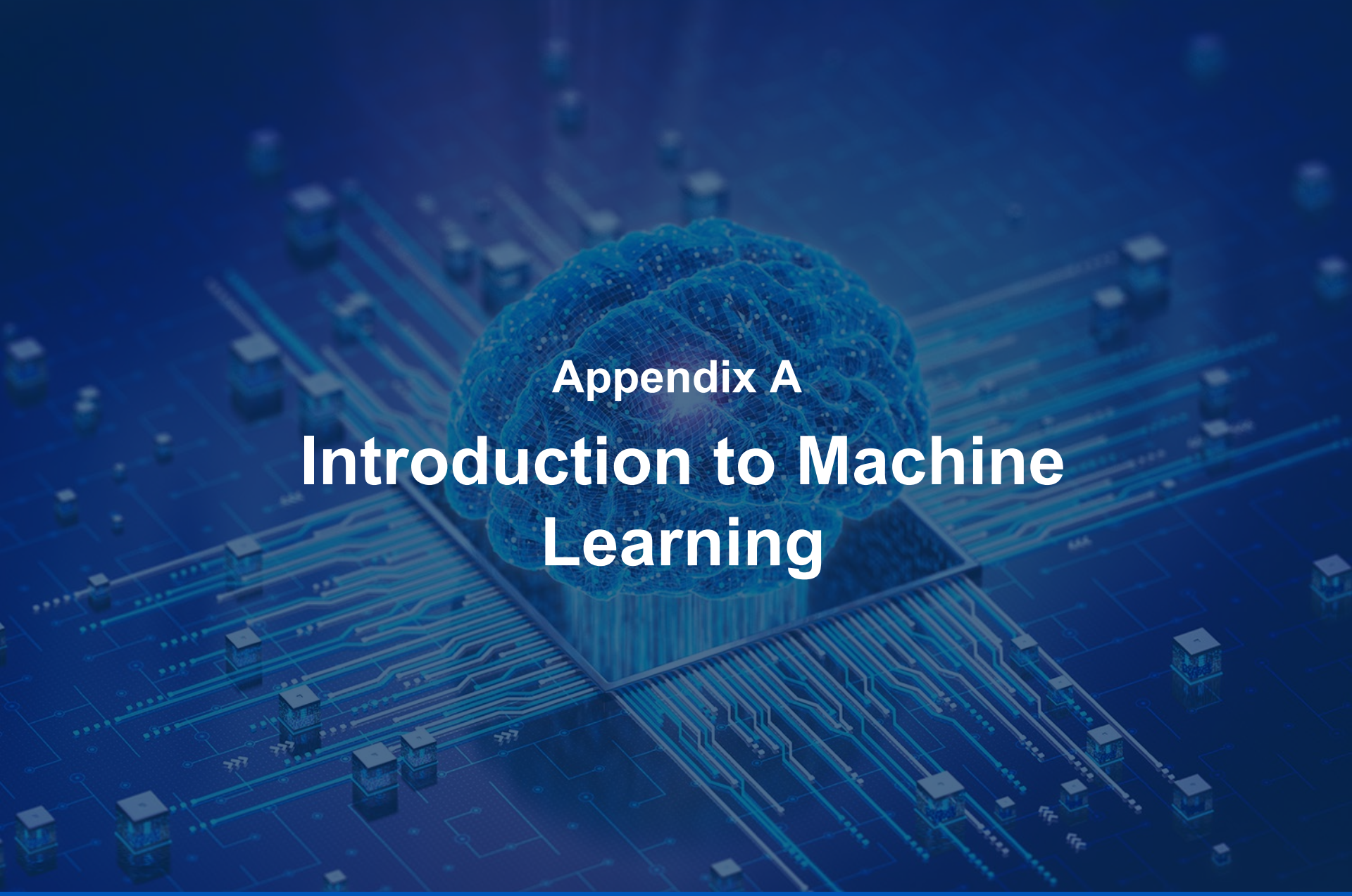
Chapter 4

Course Summary

LEARNING TREE™ INTERNATIONAL

Course Objectives

- Review Jupyter as a tool for working with Python for data analytics
- Import data from varied sources into Python
- Manipulate and transform data in preparation for data mining
- Pre-process unstructured data for data analytic tasks
- Optional: introduce machine learning in Python



Appendix A

Introduction to Machine Learning

LEARNING TREE INTERNATIONAL

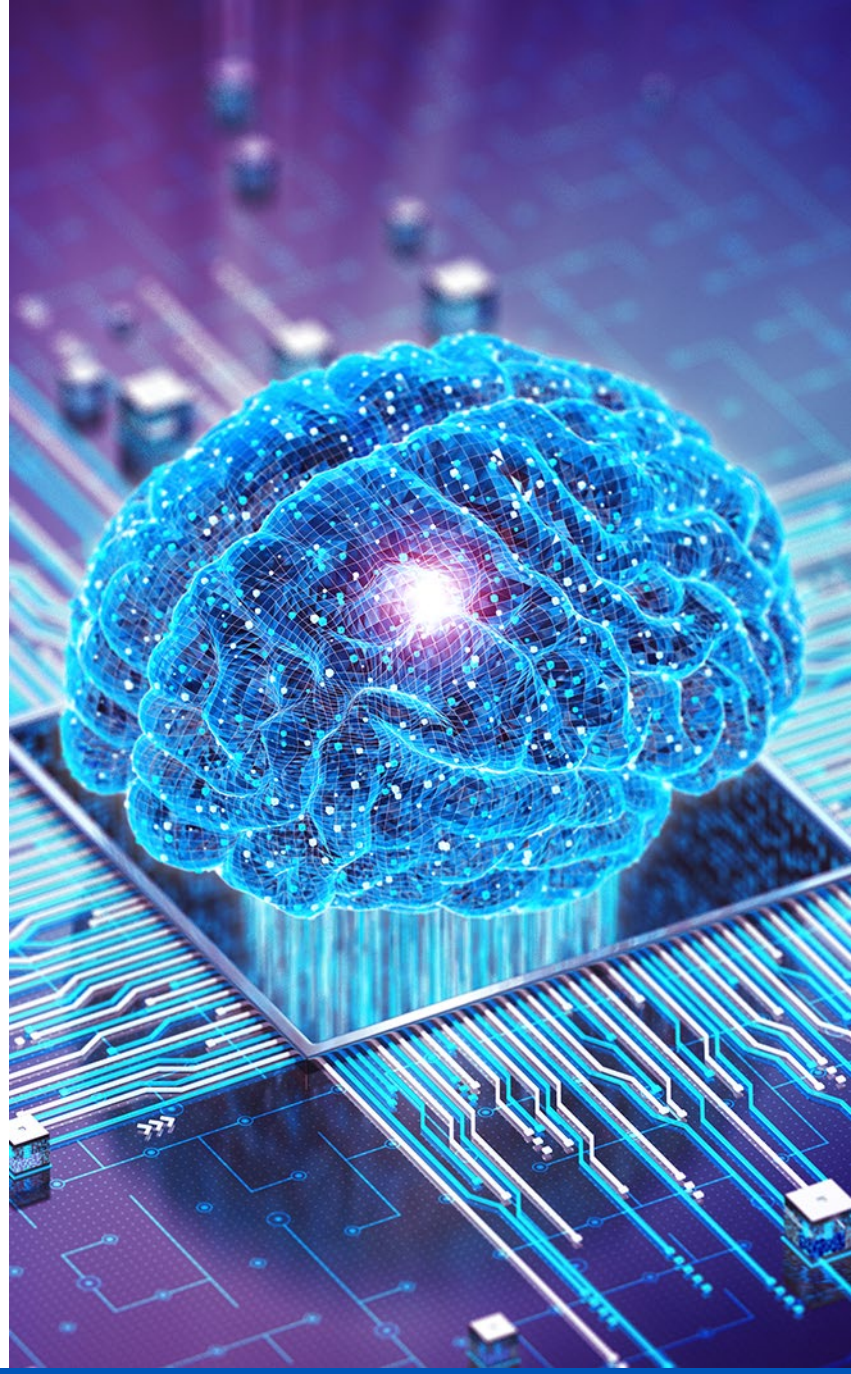
Objectives

- Introduce machine learning in Python

Contents

Machine Learning: Clustering

- K-Means Algorithm
- Hands-On Exercise A.1



Clustering: Introduction

- **Clustering is a machine learning technique that involves looking for natural groupings in data items that are *not* driven by pre-specified criteria**
  - For example, do our customers naturally fall into different behavioral groups?
- **Referred to as unsupervised learning because there is no set target**
- **Often used to aid in understanding of an aspect of the business in order to provide better products/services, marketing campaigns**
  - For example, a better understanding of mobile phone customers may lead to different network packages being offered
- **Extensively used in image segmentation, which is a crucial pre-step for object recognition**
  - Groups pixels in an image with similar characteristics together (e.g., color intensity) thus dividing the image into regions corresponding to objects or parts of objects
- **Clustering is valuable for various applications, where labeled data might be limited, or for exploring and understanding structure within the data itself**

Concept of Similarity

- **Clustering is based on the notion of similarity**
  - Challenge: how do we explain why two customers, or two companies are similar to each other?
    - A data item such as customer has many attributes; e.g., age, marital status, home ownerships, etc.
    - Once these attributes can be represented as numeric data, distance measures can be used as a measure of similarity
    - Requires data preprocessing in many cases; e.g., encoding categorical data
- **Observations within the group should be similar (shorter distance) to each other, and dissimilar (longer distance) from observations in other groups**
    - Requires understanding of the data to ensure an appropriate measure is being used
- **Can also be used as a preprocessing step for further analytics**
  - For example, clustering maybe used to define groups that are then used as target labels in a classification problem to predict new customer behavior

Distance Measures

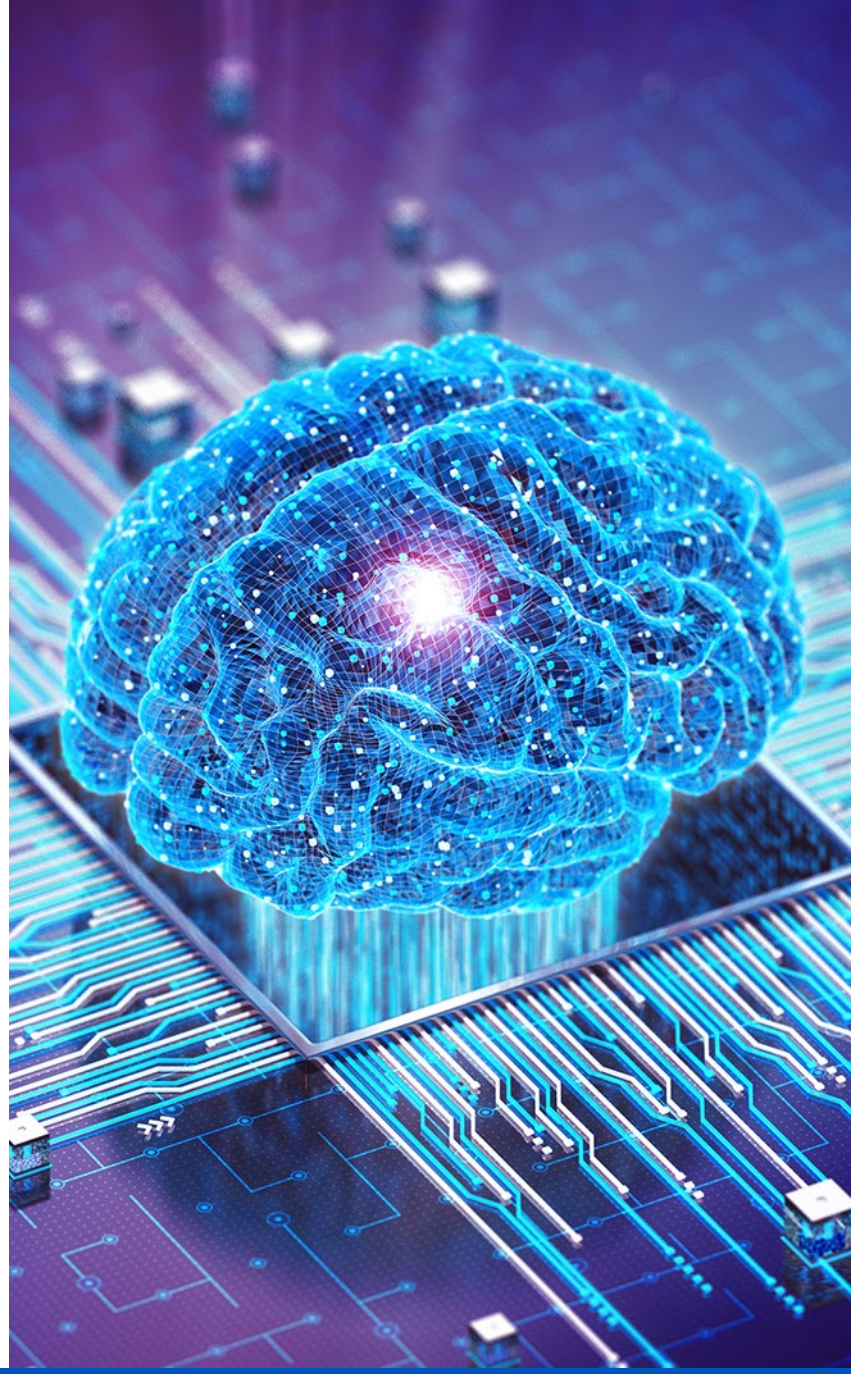
- **Euclidean distance**
  - The distance between two points that one would measure with a ruler is given by the Pythagorean theorem
- **Squared Euclidean distance**
  - The square of the Euclidean distance
- **Manhattan distance**
  - The distance between two points in a grid based on a strictly horizontal and/or vertical path
  - The sum of the horizontal and vertical components

Contents

- Machine Learning: Clustering

K-Means Algorithm

- Hands-On Exercise A.1

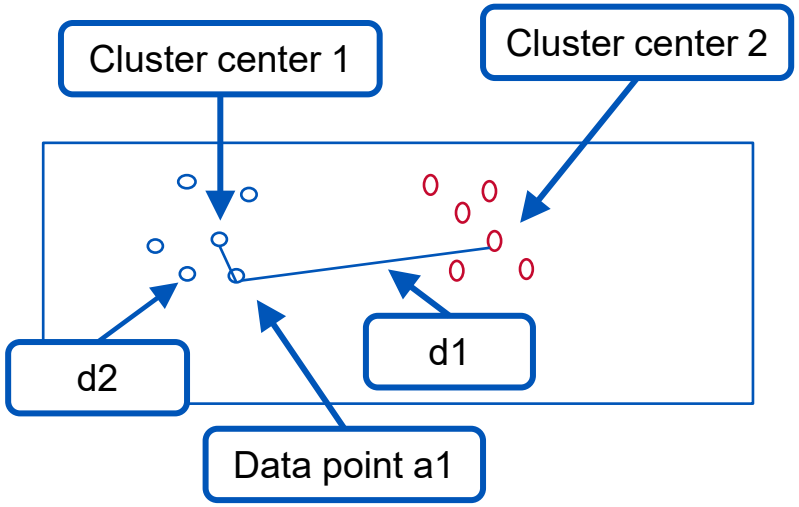


K-Means Clustering

- **K-Means is the simplest and most popular algorithm**
- **Before clustering the data, it may be necessary to rescale variables for comparison**
- **Variables that are quite different aren't comparable as distance measures**
  - For example, height and weight, which are measured in very different units (centimeters and kilograms)
  - It is good practice to z-standardize them in order to give each variable equal weight
- **K-Means is not very suited to binary values or discrete or categoric attributes**
  - K-Means computes *means*
  - The mean value is not meaningful on this kind of data

K-Means Clustering

- **Steps of the K-Means algorithm**
  - Choose the desired number of clusters (e.g., *k*=2)
  - Randomly choose *K* points to act as cluster centers
  - For each data point, compute the distance from all cluster centers
  - Assign the data point to the closest cluster center
  - Recompute the cluster centers by calculating the average of cluster members
  - Repeat until there is no more shifting of data points between clusters



Logic of allocation:
If d1 > d2, then
{a1 is part of cluster 1}
else
{a1 is part of cluster 2}

Applying K-Means

- To apply K-Means using `scikit-learn`

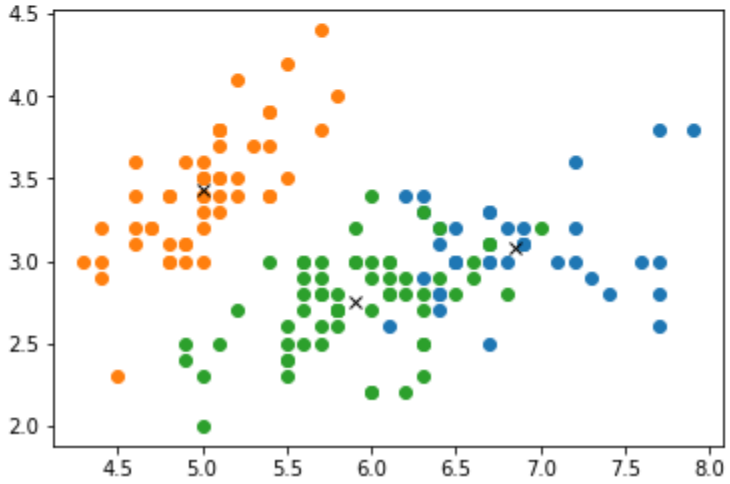
```
from sklearn import cluster

iris = pd.read_csv('iris.csv')
# Drop the species column as it is non-numeric
iris = iris.drop('species', axis=1)
iris = np.array(iris)
```

```
k_means = cluster.KMeans(n_clusters=3)
k_means.fit(iris)
```

```
KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300,
    n_clusters=3, n_init=10, n_jobs=1, precompute_distances='auto',
    random_state=None, tol=0.0001, verbose=0)
```

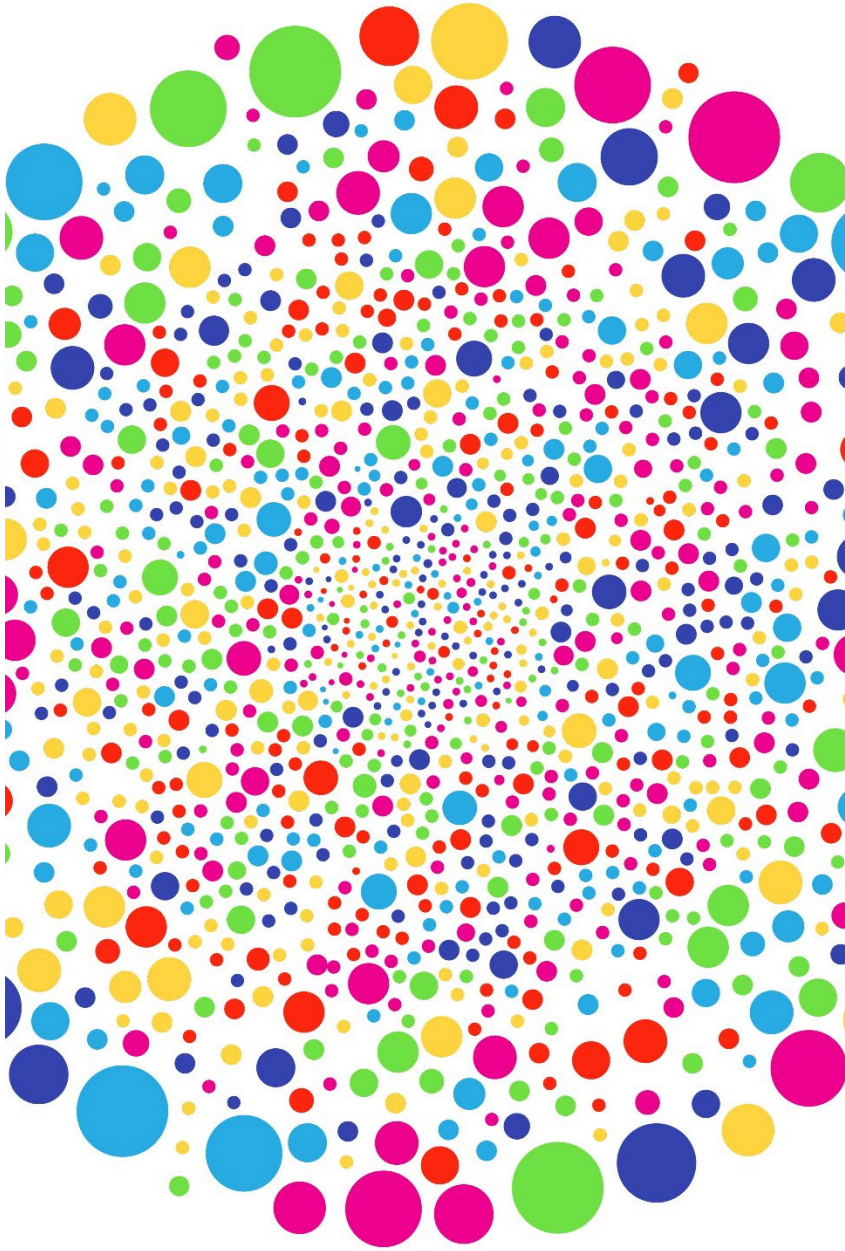
Visualizing the Clusters

```
labels = k_means.labels_
centroids = k_means.cluster_centers_

from matplotlib import pyplot

for i in range(3):
    ds = iris[np.where(labels==i)]

    # plot the data observations
    pyplot.plot(ds[:,0],ds[:,1],'o')

    # plot the centroids
    lines = pyplot.plot(centroids[i,0],centroids[i,1],'kx')
```



Limitations of K-Means Clustering

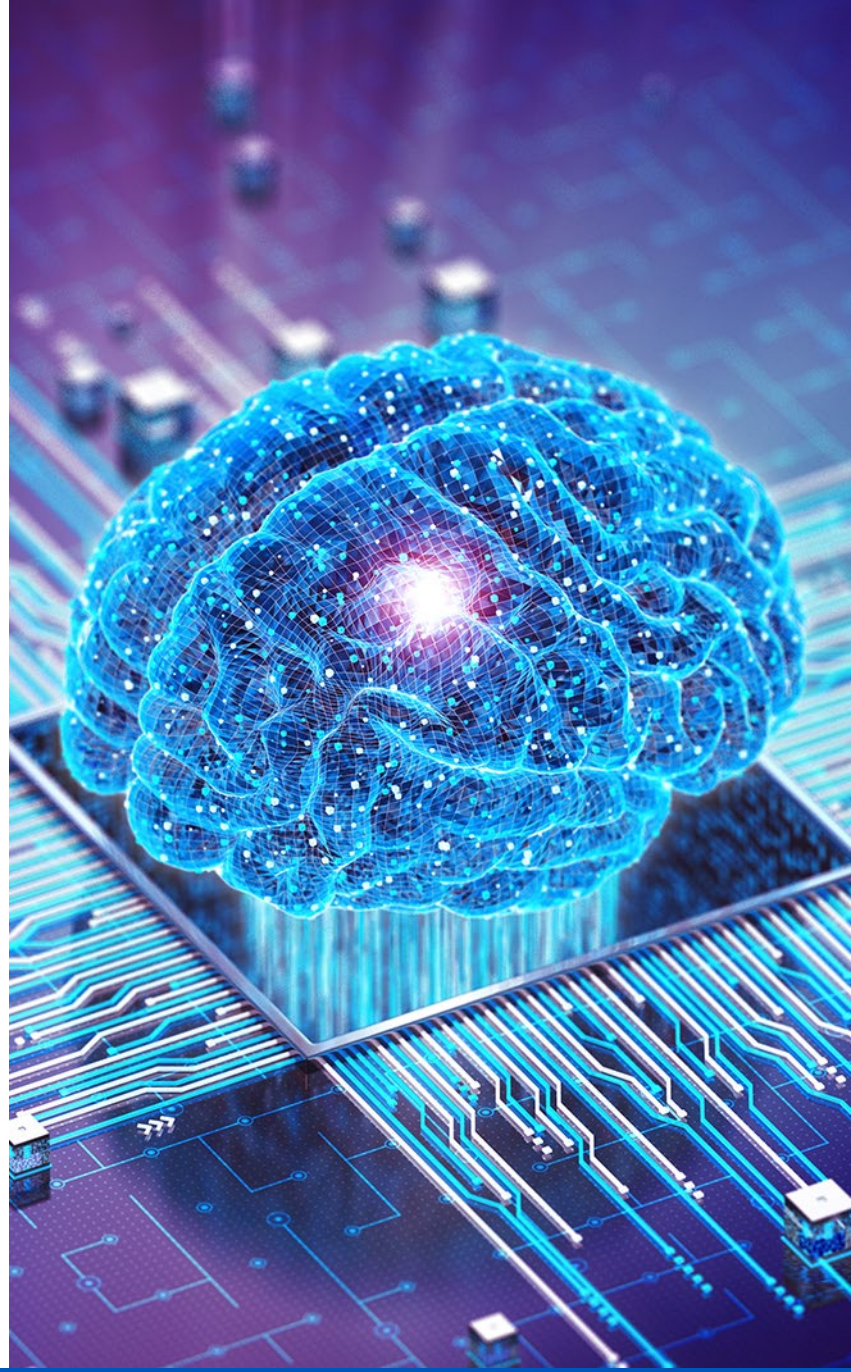
- **Initial assignment of cluster centers is random**
  - Thus, each time the algorithm is run, it may give different clusters
- **Initial number of clusters has to be predetermined**
- **Each data point is assigned to only one cluster**
  - Sometimes, there are points equidistant from two cluster centers
- **Sensitive to outliers**

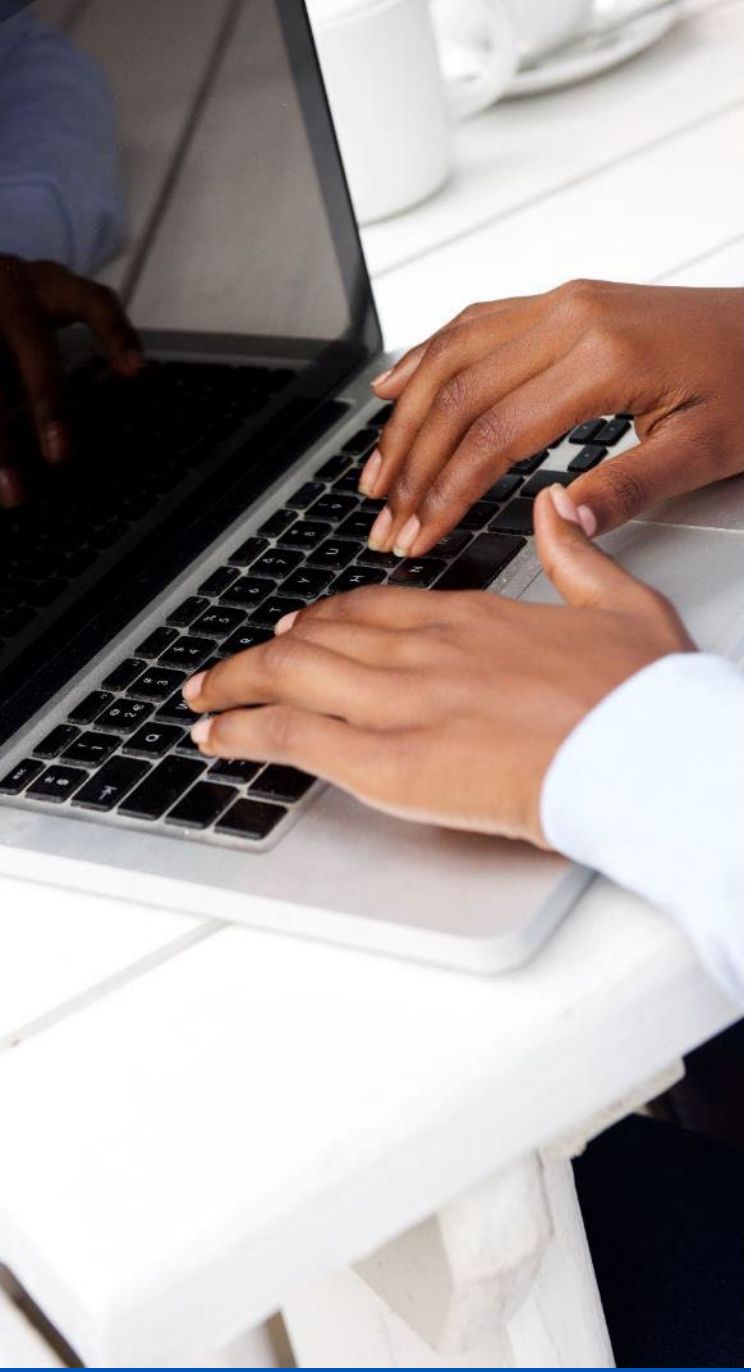


Contents

- Machine Learning: Clustering
- K-Means Algorithm

Hands-On Exercise A.1





Hands-On Exercise A.1

Jupyter Exercise Dashboard

In your Jupyter Exercise Dashboard, please refer to Hands-On Exercise A.1: Cluster Analysis on Structured Data With Python

More...

- Looking further…

  scikit-learn has many other subpackages

  e.g., sklearn.linear_model for LogisticRegression

- To explore, import the package:

  use `dir()` to get a list of names

  use `help()` to get documentation

Beware: There is often a lot of documentation

Objectives

- Introduce machine learning in Python